



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Колледж программирования и кибербезопасности

ДИПЛОМНЫЙ ПРОЕКТ

СПЕЦИАЛЬНОСТЬ 09.02.07

Информационные системы и программирование

на тему:

«Разработка веб-приложения для системы управления
библиотекой»

Выполнил студент

группы ЩПКО-02-22

подпись

Ю.А. Загородная

ФИО студента

Руководитель

подпись

Е.В. Понеделко

ФИО руководителя

Нормоконтроль

подпись

М.Л. Мымрина

ФИО контролера

Москва 2026

Утверждаю
Председатель ПЦК
«Программирование»
_____ Мымрина М.Л.
подпись *ФИО*
«___» _____ 202__г.

ЗАДАНИЕ
на дипломный проект

студенту Загородной Юлии Андреевне группы ЩПКО-02-22
по специальности 09.02.07 Информационные системы и программирование

Тема: «Разработка веб-приложения для системы управления библиотекой»

Пояснительная записка

Введение

1 Исследование предметной области

- 1.1 Анализ предметной области
- 1.2 Анализ существующих программных решений
- 1.3 Определение требований к разрабатываемому продукту

2 Проектирование веб-приложения системы управления библиотекой

- 2.1 Проектирование архитектуры приложения
- 2.2 Проектирование базы данных
- 2.3 Проектирование интерфейса пользователя

3 Разработка веб-приложения системы управления библиотекой

- 3.1 Выбор инструментальных средств разработки
- 3.2 Разработка базы данных
- 3.3 Разработка функционала веб-приложения

Заключение

Список использованных источников

Графическая часть проекта / Приложения

Приложение к заданию на дипломный проект.

В данном дипломном проекте необходимо разработать веб-приложение «КнигоЛов» для автоматизации процессов управления библиотекой, обеспечивающее учёт книжного фонда, выдачу литературы, контроль задолженностей и взаимодействие с читателями, с целью повышения эффективности работы библиотеки и удобства её пользователей.

Функции, которые должны быть реализованы в приложении:

- авторизация и регистрация пользователей в системе;
- управление данными книжного фонда (добавление, редактирование, удаление данных о книгах);
- бронирование книг;
- оформление выдачи и возврата книг;
- контроль сроков возврата;
- поиск и фильтрация книг по различным критериям;
- отображение в личном кабинете читателя текущих бронирований, штрафов.

Задание на дипломный
проект выдал

«__»_____ 2026 г.

*Подпись руководителя
проекта*

Понеделко Е.В.
*ФИО руководителя
проекта*

Задание на дипломный
проект получил

«__»_____ 2026 г.

*Подпись студента-
исполнителя проекта*

Загородная Ю.А.
*ФИО студента-
исполнителя проекта*

Срок представления к защите дипломного проекта: до «__» _____ 202__ г.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ИСПОЛЬЗОВАННЫХ ОБОЗНАЧЕНИЙ	2
ВВЕДЕНИЕ	3
1 ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.1 Анализ предметной области	5
1.2 Анализ существующих программных решений	6
1.3 Определение требований к разрабатываемому продукту	8
2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ СИСТЕМЫ УПРАВЛЕНИЯ БИБЛИОТЕКОЙ	11
2.1 Проектирование архитектуры приложения	11
2.2 Проектирование базы данных	11
2.3 Проектирование интерфейса пользователя	15
3 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ СИСТЕМЫ УПРАВЛЕНИЯ БИБЛИОТЕКОЙ	25
3.1 Выбор инструментальных средств разработки	25
3.2 Разработка базы данных	26
3.3 Разработка функционала веб-приложения.....	34
ЗАКЛЮЧЕНИЕ	51
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ИСПОЛЬЗОВАННЫХ ОБОЗНАЧЕНИЙ

- ISBN – международный стандартный книжный номер.
- ORM – технология, позволяющая разработчикам работать с реляционными базами данных посредством объектов языков программирования.

ВВЕДЕНИЕ

Современное образование характеризуется активным внедрением цифровых технологий в различные сферы деятельности, включая библиотечное дело. В условиях роста информационных потоков и повышения требований к качеству обслуживания читателей, автоматизация библиотечных процессов становится необходимостью. Городские библиотеки, библиотеки учебных заведений и музейные библиотеки требуют эффективных решений для управления книжным фондом, учета выдачи литературы и контроля читательской активности.

На сегодняшний день многие библиотеки продолжают использовать традиционные методы учета – бумажные журналы и карточные системы, что приводит к значительным временным затратам и ошибкам. Читатели не имеют оперативного доступа к информации о доступности книг, не могут отслеживать сроки возврата и состояние своих задолженностей. Это создает дополнительную нагрузку на сотрудников библиотеки и снижает качество обслуживания читателей.

Объектом исследования данной работы является процесс автоматизации деятельности библиотек различного типа.

Предметом исследования выступают методы и средства разработки веб-приложения для управления библиотечными процессами.

Целью данного проекта является разработка системы управления библиотекой «КнигоЛов», которая не только автоматизирует процессы учета книжного фонда, но и предоставляет читателям возможность в любое время получать информацию о доступности книг, отслеживать сроки возврата через личный кабинет.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- провести анализ предметной области и существующих аналогов
- определить требования к разрабатываемому продукту;

- спроектировать архитектуру, базу данных и пользовательский интерфейс веб-приложения;
- определить инструментальные средства разработки;
- разработать базу данных;
- реализовать функционал веб-приложения;
- выполнить тестирование разработанного веб-приложения.

Разрабатываемая система будет предназначена для сотрудников библиотеки и читателей, создавая единую платформу для управления всеми аспектами библиотечной работы. Результаты данной работы обеспечат оптимизацию внутренних процессов и улучшат взаимодействие между библиотекой и читателями, что повысит общий уровень удовлетворенности пользователей.

1 ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области

Предметная область данного дипломного проекта охватывает процессы автоматизации деятельности библиотек различного типа. В рамках исследования рассматриваются библиотеки учебных заведений, городские общедоступные библиотеки, а также библиотеки научных и культурных организаций. Все эти учреждения объединяет необходимость систематизации книжного фонда, учета выдачи литературы и взаимодействия с читателями.

Пользователи разрабатываемого продукта подразделяются на две категории: читатели и библиотекари. Библиотекари ведут книжный фонд, оформляют выдачу и возврат литературы, контролируют задолженности, начисляют штрафы и работают с читательскими билетами. Читателям же требуется удобный доступ к информации о наличии книг в библиотеке, возможность их бронирования и отслеживание сроков возврата и штрафов.

Особенностью данной предметной области является работа с большими объемами данных о книгах и читателях, а также необходимость обеспечения прозрачности и контроля всех библиотечных операций [4].

При анализе предметной области были выявлены ключевые проблемы в организации работы библиотек:

- ручной учет книг и читателей: многие библиотеки используют бумажные журналы учета, что приводит к ошибкам и затрудняет поиск информации;
- сложность контроля сроков возврата: отсутствие автоматизированного напоминания приводит к длительным просрочкам и потере литературы;
- ограниченный доступ к информации: читатели не могут быстро узнать о доступности нужной книги.

Основные особенности библиотечной системы, которые будут учтены в создаваемом веб-приложении:

- различные роли пользователей (читатель, библиотекарь);
- необходимость учета различных категорий литературы;
- работа с несколькими экземплярами одной книги;
- автоматическое создание штрафов при просрочке возврата;
- ограничение на максимальное количество бронирований для одного читателя (не более пяти книг);
- невозможность бронирования книг для библиотекарей;
- фиксация ответственного библиотекаря при выдаче и возврате книги;
- защита от удаления книги при наличии активных бронирований.

Разрабатываемая система направлена на решение проблем организации внутреннего учета конкретной библиотеки через создание специализированного веб-приложения, учитывающего специфику работы библиотек и предоставляющего необходимый функционал для автоматизации ключевых бизнес-процессов.

Создание специализированного решения для библиотек является экономически целесообразным и будет способствовать повышению эффективности их работы за счет снижения трудозатрат, минимизации потерь литературы и улучшения качества обслуживания читателей.

1.2 Анализ существующих программных решений

На сегодняшний день существует множество программных продуктов, решающих задачу автоматизации библиотек, каждый из которых обладает своими преимуществами и недостатками. Для проведения анализа были выбраны два наиболее используемых в библиотеках приложения, представленных на российском рынке.

Одной из самых известных автоматизированных библиотечных систем в России является приложение «ИРБИС». Данный продукт предназначен для комплексной автоматизации библиотечных процессов и широко используется

в различных типах библиотек. Система обладает мощным средством составления каталогов и развитыми средствами научной обработки документов, позволяя вести электронный каталог, организовывать книговыдачу и формировать отчетность. Однако, несмотря на богатые возможности, приложение имеет существенные недостатки. Пользователи отмечают сложность в освоении и обслуживании системы, устаревшее оформление, ориентированное преимущественно на специалистов, а также отсутствие полноценного личного кабинета для читателей, что ограничивает возможности удаленного взаимодействия.

Другим распространенным решением является приложение «1С:Библиотека», созданное на платформе «1С:Предприятие» и предназначенное для автоматизации библиотек учебных заведений. К достоинствам данной системы можно отнести возможность взаимодействия с другими продуктами «1С:Предприятие», ведение учета в соответствии с установленными стандартами, а также средства учета обеспеченности учебной литературой. Вместе с тем, программа является коммерческим продуктом, требующим приобретения лицензий, что увеличивает стоимость внедрения. Возможности системы могут оказаться избыточными для небольших библиотек, либо, напротив, ограниченными рамками платформы, что существенно затрудняет приспособление под нужды конкретного учреждения.

Проведенный анализ существующих программных продуктов позволяет сделать вывод о целесообразности разработки нового веб-приложения «КнигоЛов». В отличие от рассмотренных аналогов, разрабатываемая система будет в наибольшей степени учитывать потребности как библиотекарей, так и читателей, сочетая простоту использования с необходимыми возможностями. Создание нового приложения позволит не только преодолеть недостатки существующих решений, но и улучшить читательский опыт, предоставив посетителям библиотеки современные возможности удаленного

взаимодействия с библиотекой, а сотрудникам – удобные способы автоматизации повседневных задач.

1.3 Определение требований к разрабатываемому продукту

Требования к программному продукту принято разделять на две большие группы: функциональные, описывающие действия, которые система должна выполнять, и нефункциональные, определяющие ее качественные характеристики и ограничения.

Функциональные требования описывают, что система должна делать. Они определяют конкретные функции и задачи, которые система должна выполнять для удовлетворения потребностей пользователя или бизнеса. Обычно функциональные требования касаются взаимодействия пользователя с системой и её поведения в различных ситуациях [6].

На основе анализа предметной области были определены следующие функциональные требования к разрабатываемому программному продукту:

- пользователь должен иметь возможность зарегистрироваться в системе, указав фамилию, имя, адрес электронной почты, номер телефона и пароль;
- при регистрации должна выполняться проверка корректности введенных данных;
- система не должна допускать регистрацию с уже существующим адресом электронной почты;
- пользователь должен иметь возможность войти в систему, введя адрес электронной почты и пароль;
- при вводе неверных учетных данных система должна уведомлять пользователя об ошибке;
- в профиле читателя должны отображаться сведения о пользователе: фамилия, имя, номер телефона, адрес электронной почты;
- библиотекарь должен иметь возможность изменять сведения о книге;

- библиотекарь должен иметь возможность добавлять новые книги в систему;
- библиотекарь должен иметь возможность удалять данные книги при условии отсутствия активных бронирований;
- система должна автоматически уменьшать количество доступных экземпляров при бронировании книги;
- система должна автоматически увеличивать количество доступных экземпляров при возврате книги;
- любой посетитель должен иметь возможность просматривать список книг;
- система не должна позволять читателю забронировать более пяти книг одновременно;
- система не должна позволять библиотекарю бронировать книги;
- при возврате книги после установленного срока система должна автоматически создавать запись о денежном взыскании;
- читатель должен иметь возможность просматривать в личном кабинете список активных бронирований, выданных книг с указанием сроков возврата и список активных денежных взысканий;
- библиотекарь должен иметь возможность просматривать список всех бронирований с фильтрацией по статусу;
- система должна автоматически проверять просроченные выдачи и создавать записи о взысканиях;

Помимо функциональных требований, важное значение имеют нефункциональные требования. Они описывают, как должна работать система, и не связаны с её конкретными функциями. Эти требования касаются качественных характеристик системы и её работы в определённых условиях. Их цель – обеспечить удобство, производительность и стабильность системы.

Были определены следующие нефункциональные требования:

- интерфейс должен иметь единый стиль оформления;
- пароли должны храниться в преобразованном виде;

- система должна предоставлять сообщения об успешном выполнении действий или об ошибках;
- страницы должны правильно отображаться на разных экранах
элементы управления на мобильных устройствах должны быть достаточного размера для удобного нажатия;
- доступ к страницам библиотекаря должен быть закрыт для обычных читателей;
- система должна корректно обрабатывать отсутствие запрашиваемых страниц и внутренние ошибки;
- сервер должен работать под управлением операционной системы Windows с установленной средой выполнения Python версии 3.8 или выше;
- для работы с приложением необходим современный веб-обозреватель, поддерживаются последние версии «Chrome», «Firefox», «Microsoft Edge»;
- устройство пользователя должно иметь экран с минимальным разрешением 420 пикселей по ширине и подключение к сети.

Таким образом, в результате проведенного анализа были определены функциональные и нефункциональные требования к веб-приложению «КнигоЛов», охватывающие все основные направления работы библиотеки. Сформулированные требования полностью учитывают особенности библиотечной деятельности, создавая основу для дальнейшего проектирования архитектуры, базы данных и пользовательского интерфейса.

2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ СИСТЕМЫ УПРАВЛЕНИЯ БИБЛИОТЕКОЙ

2.1 Проектирование архитектуры приложения

Структура программного комплекса включает в себя два основных компонента: веб-приложение и база данных. Для корректной работы программного комплекса необходимо определить их взаимодействие между собой и настроить обмен данными. Для обращения веб-приложения к базе данных предполагается использование объектно-реляционного отображения ORM. ORM (Object-Relational Mapping) – технология, позволяющая разработчикам работать с реляционными БД посредством объектов языков программирования. Она связывает два мира: мир объектов и мир таблиц, столбцов и строк, присущих реляционным базам данных [8]. Схема структуры веб-приложения представлена на рисунке 1.

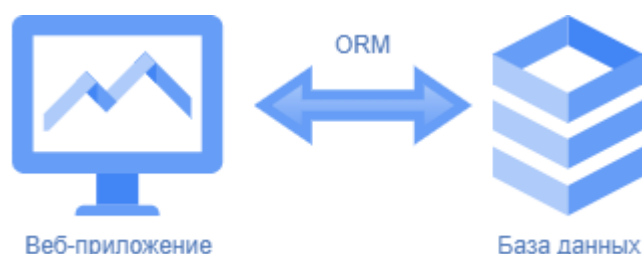


Рисунок 1 – Структура веб-приложения системы управления библиотекой

Взаимодействие между компонентами происходит следующим образом: веб-приложение обрабатывает HTTP-запросы от пользователей. Для выполнения операций с базой данных используется ORM-слой, который преобразует обращения к объектам в SQL-инструкции и возвращает полученные данные в виде объектов.

2.2 Проектирование базы данных

После анализа предметной области было установлено то, что необходимо создание базы данных для хранения информации книжного фонда библиотеки.

Логическая структура базы данных – это концептуальный уровень организации данных, определяющий, как информация представлена, связана и обрабатывается внутри системы управления базами данных. Это ключевой аспект, обеспечивающий удобство работы пользователей и гибкость управления данными [7].

Разработка логической структуры базы данных делится на инфологическое и даталогическое проектирование.

На этапе инфологического проектирования была выбрана модель «Сущность-Связь», которая подразумевает построение ER-диаграммы, которая отражает предметную область. ER-диаграмма играет ключевую роль в проектировании баз данных, так как она наглядно иллюстрирует структуру данных и взаимосвязи между сущностями, что помогает четко определить их атрибуты и типы отношений.

Исходя из анализа предметной области были определены сущности: «Пользователи», «Книги», «Бронирования», «Штрафы», «Авторы», «Жанры», «Издательства».

Сущность «Пользователи» отражает информацию о всех зарегистрированных в системе пользователях и имеет следующие атрибуты: идентификатор пользователя, адрес электронной почты, номер телефона, зашифрованный пароль, фамилия, имя, роль (библиотекарь, читатель), дата регистрации, статус.

Сущность «Книги» отражает информацию об изданиях, имеющихся в библиотечном книжном фонде, и имеет следующие атрибуты: идентификатор книги, международный стандартный книжный номер (ISBN), название книги, описание, год издания, общее количество экземпляров, количество доступных для выдачи экземпляров, изображение обложки.

Сущность «Бронирования» отражает информацию о процессе бронирования, выдачи и возврата книг и имеет следующие атрибуты:

- идентификатор бронирования;
- статус бронирования (активно, выдано, закрыто);

- дата и время создания бронирования;
- дата и время выдачи книги;
- планируемая дата возврата;
- дата и время фактического возврата.

Связь между «Бронированиями» и «Пользователями» (читатель) — «многие-к-одному»: один читатель может иметь несколько бронирований, но каждое бронирование принадлежит только одному читателю.

Связь между «Бронированиями» и «Пользователями» (библиотекарь, выдавший книгу) — «многие-к-одному»: один библиотекарь может выдать множество книг, но каждое бронирование имеет только одного библиотекаря, выдавшего книгу.

Связь между «Бронированиями» и «Пользователями» (библиотекарь, принявший возврат) — «многие-к-одному»: один библиотекарь может принять множество книг, но каждое бронирование имеет только одного библиотекаря, принявшего возврат.

Связь между «Бронированиями» и «Книгами» — «многие-к-одному»: одна книга может быть забронирована несколько раз, но каждое бронирование относится только к одному экземпляру книги.

Сущность «Штрафы» отражает информацию о денежных взысканиях за просрочку возврата книг и имеет следующие атрибуты:

- идентификатор штрафа;
- сумма штрафа;
- причина начисления;
- статус штрафа (активен, оплачен);
- дата начисления;
- дата оплаты.

Связь между «Штрафами» и «Пользователями» — «многие-к-одному»: один читатель может иметь несколько штрафов, но каждый штраф принадлежит только одному читателю.

Связь между «Штрафами» и «Бронированиями» — «один-к-одному»: один штраф соответствует одному бронированию, и каждое бронирование может иметь только один штраф (или не иметь его вовсе).

Сущность «Авторы» отражает информацию об авторах книг и имеет следующие атрибуты: идентификатор автора, фамилия, имя, отчество. Связь между сущностями «Книги» и «Авторы» — «многие-ко-многим»: одна книга может быть написана несколькими авторами, и один автор может написать несколько книг.

Сущность «Жанры» отражает информацию о литературных жанрах и имеет следующие атрибуты: идентификатор жанра, наименование жанра. Связь между «Книгами» и «Жанрами» — «многие-ко-многим»: одна книга может относиться к нескольким жанрам, и один жанр может включать множество книг.

Сущность «Издательства» отражает информацию об издательствах, выпускающих книги, и имеет следующие атрибуты: идентификатор издательства, наименование издательства. Связь между «Книгами» и «Издательствами» — «многие-ко-многим»: одна книга может быть издана несколькими издательствами, и одно издательство может выпускать множество книг.

Результатом инфологического проектирования базы данных является ER-диаграмма, визуализирующая связи между сущностями и их атрибутами, которая представлена в Графической части. Для проектирования ER-диаграммы было выбрано средство SmartDraw в связи с удобством использования и с наличием нужных функций для проектирования.

На втором этапе проектирования сущности «Пользователи», «Книги», «Бронирования», «Штрафы», «Авторы», «Жанры», «Издательства» преобразованы в таблицы реляционной базы данных «users», «books», «reservations», «fines», «authors», «genres», «publishers» соответственно [2].

Атрибуты преобразованы в столбцы таблиц с определенными типами данных. Связи «один-ко-многим» реализованы с помощью внешних ключей.

Связи «многие-ко-многим» реализованы с помощью таблиц «book_authors», «book_genres», «book_publishers». Определены типы данных и ограничения целостности.

Результатом даталогического проектирования базы данных является реляционная схема, представленная на рисунке 2.

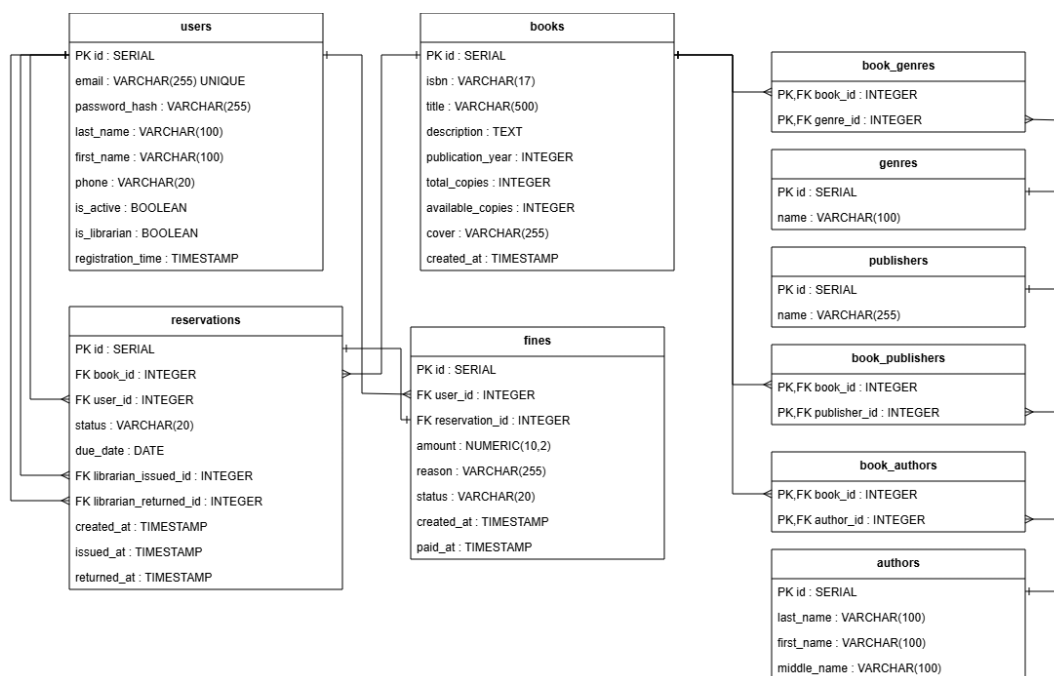


Рисунок 2 – Реляционная схема базы данных

Проектирование реляционной схемы проводилось в онлайн-инструменте SmartDraw, который предоставляет удобные средства для визуализации структуры базы данных.

2.3 Проектирование интерфейса пользователя

Проектирование пользовательского интерфейса веб-приложения осуществлялось с использованием сервиса «draw.io».

Сервис «draw.io» представляет собой бесплатный онлайн-инструмент для создания диаграмм и макетов интерфейсов. Он оснащен необходимым инструментарием для построения схем навигации и визуального представления страниц [9].

Интерфейс проектируется с учетом разделения ролей пользователей. Для читателей и библиотекарей предусматриваются различные наборы функций и элементов управления.

Для всех категорий пользователей (гость, читатель, библиотекарь) в верхней панели навигации присутствуют общие элементы: логотип, поисковая строка и ссылка на страницу со списком книг.

Для гостя дополнительно отображаются кнопки «Вход» и «Регистрация», позволяющие авторизоваться в системе или создать новую учетную запись. Верхняя панель навигации гостя представлена на рисунке 3.

Книги Вход Регистрация

Рисунок 3 – Верхняя панель навигации гостя

Для авторизованного читателя в верхней панели навигации отображаются кнопки «Мои бронирования», «Личный кабинет» и «Выход». Это позволяет читателю отслеживать свои активные бронирования, управлять личными данными и завершать сеанс работы. Верхняя панель навигации читателя представлена на рисунке 4.

Книги Мои бронирования Личный кабинет Выход

Рисунок 4 – Верхняя панель навигации читателя

Для авторизованного библиотекаря в верхней панели навигации отображаются кнопки «Добавить книгу», «Панель управления», «Бронирования» и «Выход». Это обеспечивает библиотекарю доступ к основным функциям управления книжным фондом, просмотру статистики и работе со всеми бронированиями системы. Верхняя панель навигации библиотекаря представлена на рисунке 5.

Книги Панель управления Бронирования Выход

Рисунок 5 – Верхняя панель навигации библиотекаря

Главная страница служит точкой входа в приложение. В верхней части страницы располагается панель навигации, содержащая логотип, название системы, кнопку, ведущую на страницу книг, а также кнопки навигации,

которые варьируются в зависимости от роли пользователя. В блоке «Новинки» отображаются карточки последних добавленных книг. Каждая карточка содержит изображение обложки, название книги и кнопку «Подробнее» для перехода к детальной информации. Вайрфрейм (схематичное представление пользовательского интерфейса) главной страницы представлен на рисунке 6.

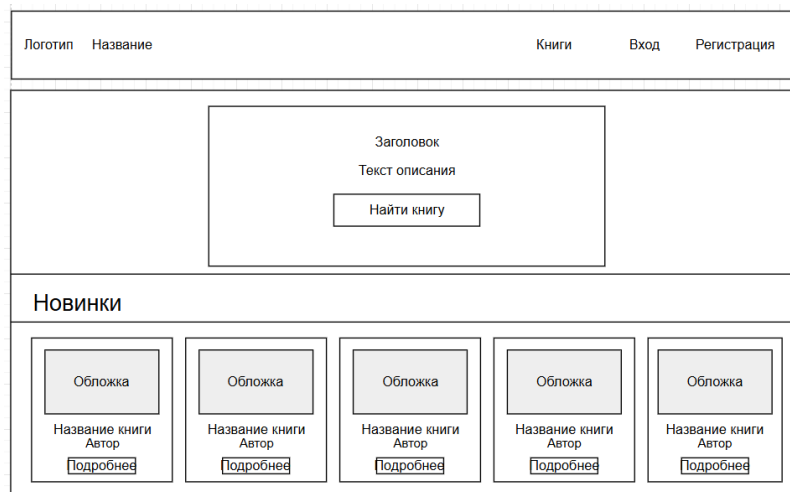


Рисунок 6 – Вайрфрейм главной страницы

Страница регистрации предназначена для создания новой учетной записи читателя. В верхней части страницы располагается панель навигации. В центральной части страницы размещена форма регистрации, содержащая поля для ввода фамилии, имени, адреса электронной почты, номера телефона и пароля, а также кнопку «Зарегистрироваться». В нижней части формы расположена ссылка «Уже есть аккаунт? Войти» для перехода на страницу авторизации. Вайрфрейм страницы регистрации представлен на рисунке 7.

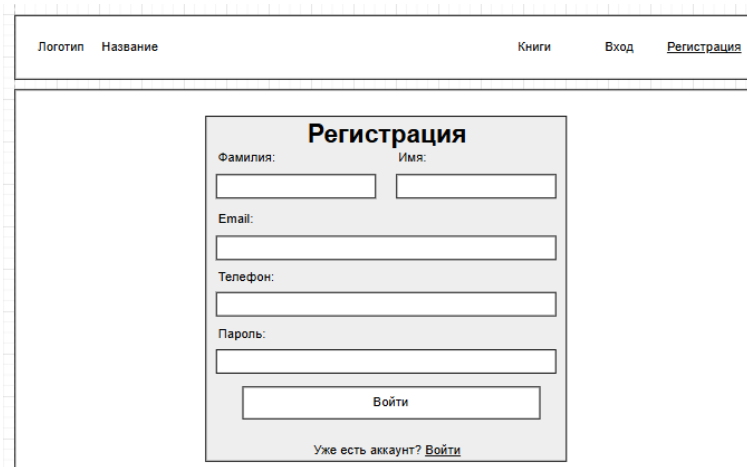


Рисунок 7 – Вайрфрейм страницы регистрации

Страница авторизации предназначена для входа зарегистрированных пользователей в систему. В верхней части страницы располагается панель навигации. В центральной части страницы размещена форма авторизации, содержащая поля для ввода адреса электронной почты и пароля, опцию «Запомнить меня» для сохранения сеанса, а также кнопку «Войти». В нижней части формы расположена ссылка «Нет аккаунта? Зарегистрироваться» для перехода на страницу регистрации. Вайрфрейм страницы авторизации представлен на рисунке 8.

Логотип Название Книги Вход Регистрация

ВХОД

Email:

Пароль:

☐ Запомнить меня

Нет аккаунта? [Зарегистрироваться](#)

Рисунок 8 – Вайрфрейм страницы авторизации

Страница каталога книг предназначена для поиска и фильтрации книг в библиотечном книжном фонде. Сверху размещена панель поиска и фильтрации, содержащая поле для поиска по названию или ISBN, выпадающие списки для выбора автора и жанра, а также маркер выбора «Только доступные» для фильтрации книг, имеющих в наличии. Рядом расположены кнопки «Применить» для выполнения фильтрации и «Сброс» для очистки всех фильтров. Основная часть страницы заполняется карточками книг. Каждая карточка содержит изображение обложки, название книги, автора и кнопку «Подробнее» для перехода к детальной информации. Вайрфрейм страницы каталога книг представлен на рисунке 9.

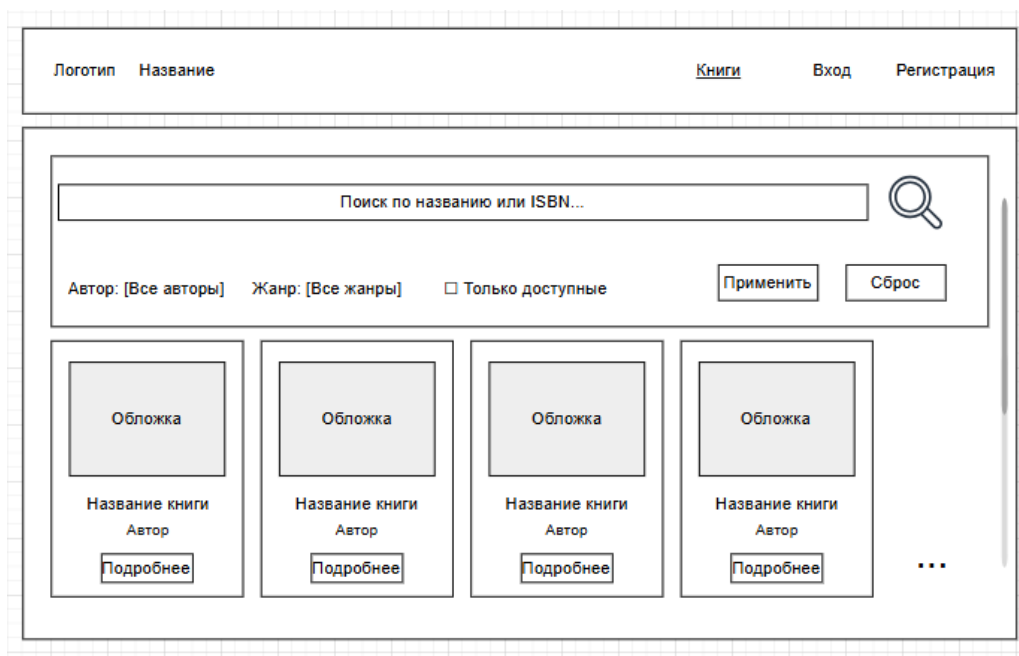


Рисунок 9 – Вайрфрейм страницы каталога книг

Страница детальной информации о книге предназначена для просмотра читателем полных сведений об издании. Данная страница открывается при нажатии на кнопку «Подробнее» в карточке книги на странице списка книг.

В центральной части страницы размещена карточка книги. В левой части карточки располагается изображение обложки. В правой части – название книги и информация о доступных экземплярах. Ниже представлена таблица с основными характеристиками книги, включающая следующие строки: автор, жанры, издательство, ISBN, год издания, общее количество экземпляров и количество доступных экземпляров. Далее расположен блок с описанием (аннотацией) книги.

В нижней части страницы находятся две кнопки: «Забронировать» (активна при наличии доступных экземпляров) и «Назад к каталогу» для возврата к списку книг. В зависимости от роли кнопка «Забронировать» меняется. Для неавторизованного пользователя кнопка отображается как «Войдите для бронирования». Для читателя кнопка остается в таком же виде, а для сотрудника библиотеки кнопка меняется на «Редактировать». Вайрфрейм страницы представлен на рисунке 10.

Авторы	ФИО
Жанры	
Издательства	
ISBN	***_**_*****_*
Год издания	
Всего экземпляров	*
Доступно экземпляров	*

Рисунок 10 – Вайрфрейм страницы детальной информации о книге

Страница детальной информации о книге в режиме библиотекаря содержит форму для просмотра и редактирования сведений об издании. Вайрфрейм страницы представлен на рисунке 11.

Рисунок 11 – Вайрфрейм страницы добавления книги

В верхней части страницы добавления книги располагается панель навигации. Основная часть страницы разделена на несколько блоков. Блок «Основная информация» содержит поля для ввода названия книги, ISBN, года издания, количества экземпляров, количества доступных экземпляров и ссылки на обложку. Блок «Авторы» содержит список выбранных авторов и кнопку «Добавить автора» для добавления нового автора в базу данных. Блок

«Жанры» и блок «Издательства» также содержат подобные кнопки. В нижней части страницы расположены кнопки «Сохранить книгу» и «Отмена».

Панель управления предназначена для просмотра библиотекарем сводной статистики работы библиотеки и управления основными процессами. Основная часть страницы содержит три статистические карточки, расположенные горизонтально. Карточки отображают общее количество книг в фонде, количество зарегистрированных читателей и количество активных бронирований. Справа от статистических карточек расположены две кнопки: кнопка «Добавить книгу» для перехода на страницу добавления новой книги в библиотечный книжный фонд и кнопка «Бронирования» для перехода к полному списку всех бронирований. В нижней части страницы размещена таблица с информацией о последних бронированиях. Таблица содержит следующие столбцы: идентификатор бронирования (ID), название книги, читатель, статус, дата и действия. В столбце «Действия» для каждого бронирования предусмотрены кнопки управления (например, «Выдать книгу» или «Принять возврат»). Вайрфрейм панели управления представлен на рисунке 12.

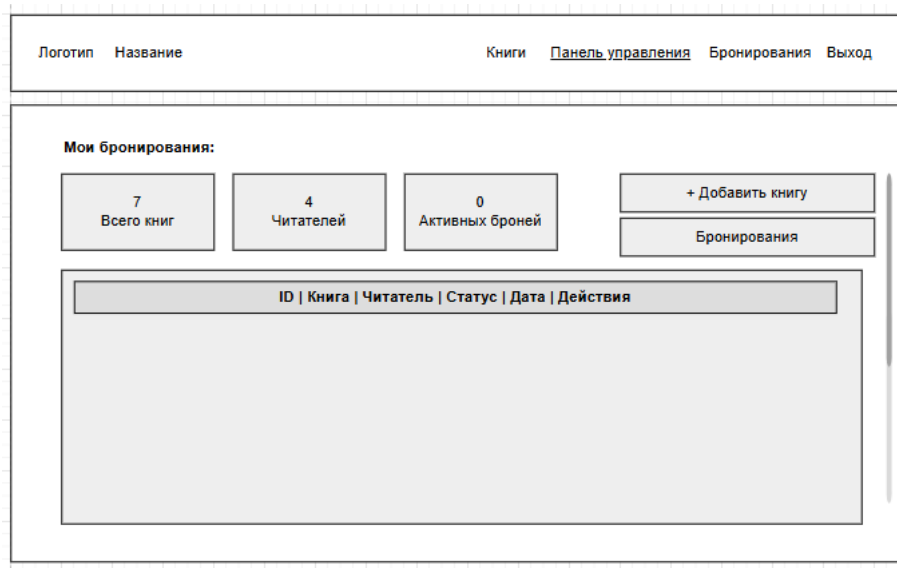


Рисунок 12 – Вайрфрейм панели управления

Страница «Мои бронирования» предназначена для просмотра читателем своих бронирований и выданных книг. Основная часть страницы содержит

список бронирований. Каждое бронирование представлено в виде отдельного блока с номером и статусом. В блоке отображается название книги, дата создания бронирования, дата выдачи, срок возврата и дата фактического возврата (если книга возвращена). В нижней части каждого блока расположена кнопка «к книге» для перехода к детальной информации об издании. Вайрфрейм страницы представлен на рисунке 13.

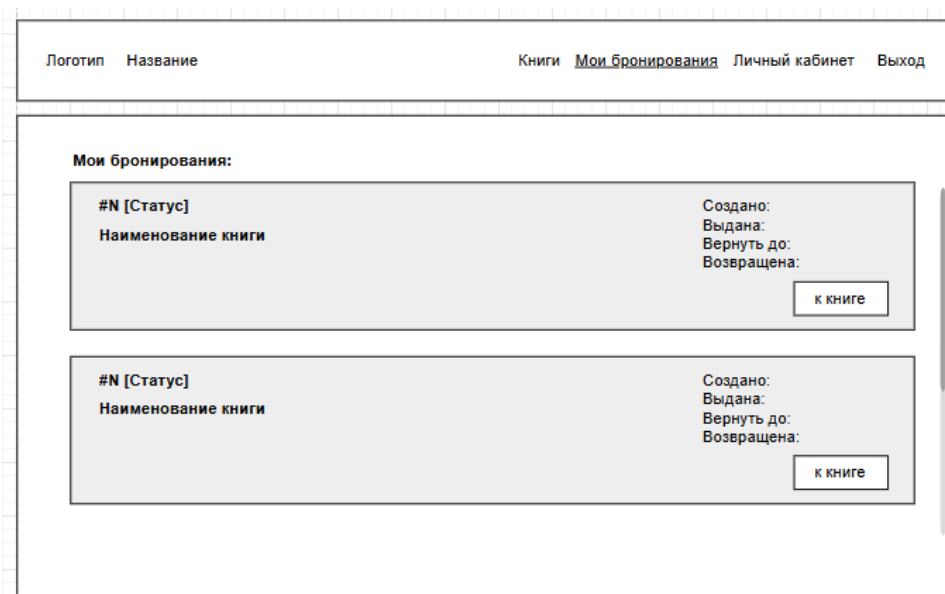


Рисунок 13 – Вайрфрейм страницы «Мои бронирования»

Страница личного кабинета читателя объединяет всю информацию о взаимодействии читателя с библиотекой. В левой верхней части страницы отображается информация о пользователе: фамилия, имя и адрес электронной почты. Ниже расположены три блока:

- блок «Активные бронирования» отображает список книг, ожидающих выдачи;
- блок «Выданные книги» отображает список книг на руках с указанием сроков возврата;
- блок «Активные штрафы» отображает непогашенные задолженности.

В нижней части страницы расположен блок «История бронирования» с таблицей, содержащей информацию о последних возвращенных книгах.

Таблица включает столбцы: название книги, дата выдачи, дата возврата и статус. Вайрфрейм страницы представлен на рисунке 14.

Логотип	Название	Книги	Мои бронирования	Личный кабинет	Выход				
ФИО электронная почта									
Активные бронирования:									
Выданные книги:									
Активные штрафы:									
История бронирования:									
<table border="1"><thead><tr><th>Книга</th><th>Дата выдачи</th><th>Дата возврата</th><th>Статус</th></tr></thead></table>						Книга	Дата выдачи	Дата возврата	Статус
Книга	Дата выдачи	Дата возврата	Статус						

Рисунок 14 – Вайрфрейм страницы личного кабинета читателя

При проектировании интерфейса учитывались следующие принципы:

- единый стиль оформления – все страницы будут использовать общую стилистику, что обеспечит визуальное единство приложения;
- адаптивная верстка – интерфейс должен корректно отображаться на устройствах с различным разрешением экрана;
- информативные сообщения – система будет использовать всплывающие уведомления для информирования пользователя о результатах операций (например, сообщение об успешном входе в систему);
- контекстная навигация – в зависимости от роли пользователя должны отображаться только доступные ему пункты меню;
- визуальная обратная связь – просроченные книги планируется подсвечивать в списках, статусы бронирований отображать цветовыми индикаторами.

При проектировании цветового оформления веб-приложения «КнигоЛов» учитывались требования к читаемости, визуальному комфорту и соответствию тематике библиотечного сервиса.

Для приложения предусмотрена светлая тема. Она воспринимается пользователями как классическая и профессиональная, она также подходит для длительной работы. Задний фон пыльно-розовый, а кнопки будут выделены, для акцентирования внимания.

Отсутствие ярких, раздражающих цветов поможет снизить утомляемость глаз. Все страницы будут выполнены в одной цветовой гамме.

3 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ СИСТЕМЫ УПРАВЛЕНИЯ БИБЛИОТЕКОЙ

3.1 Выбор инструментальных средств разработки

Для создания веб-приложения «КнигоЛов» потребовалось подобрать такой набор технологий, который бы обеспечил полноценную реализацию всех требуемых функций и высокую надёжность.

Основным языком программирования для написания серверной логики стал Python. Этот язык давно зарекомендовал себя в сфере веб-разработки благодаря лаконичному синтаксису, большому количеству готовых библиотек и активному сообществу. Использование Python позволяет быстро создавать работающие прототипы и постепенно наращивать функциональность без существенных временных затрат [3].

В качестве веб-фреймворка выбран Flask. Это легковесное решение, которое предоставляет всё необходимое для обработки HTTP-запросов, маршрутизации и работы с сессиями [1].

Для взаимодействия с базой данных используется расширение Flask-SQLAlchemy. Оно добавляет в фреймворк Flask поддержку объектно-реляционного отображения, благодаря чему разработчик может работать с записями базы данных как с обычными объектами языка программирования Python. Это избавляет от необходимости писать SQL-запросы вручную и снижает вероятность ошибок при обращении к данным.

Системой управления базами данных выбрана PostgreSQL – объектно-реляционная система управления базами данных. Данная СУБД обладает высокой надёжностью, поддерживает работу с большими объёмами информации и предоставляет возможности по обеспечению целостности данных на уровне ограничений и транзакций [5].

Сервер PostgreSQL может обслуживать одновременно несколько подключений клиентов. Для этого он запускает отдельный процесс для

каждого подключения. Таким образом, главный серверный процесс всегда работает и ожидает подключения клиентов [10].

Клиентская часть приложения строится на стандартных веб-технологиях – языке гипертекстовой разметки HTML и языке каскадных таблиц стилей CSS. Язык HTML обеспечивает структуру и определяет содержание страниц, а язык CSS – контролирует визуальное оформление, включая адаптивную вёрстку для различных устройств. Весь динамический функционал, такой как обработка форм, авторизация пользователей и бронирование книг, реализован на стороне сервера средствами языка программирования Python и веб-фреймворка Flask.

В качестве среды разработки используется GigaIDE. Данная среда предоставляет удобный интерфейс для управления файлами проекта, автодополнение кода, подсветку синтаксиса и встроенные средства запуска веб-приложений, что позволяет сосредоточиться на реализации функциональности без необходимости настройки дополнительного программного обеспечения.

Выбранный стек технологий – язык программирования Python, веб-фреймворк Flask, СУБД PostgreSQL, язык разметки HTML, язык каскадных таблиц стилей CSS и среда разработки GigaIDE – обеспечивает полный цикл разработки веб-приложения. Данные инструменты сочетают в себе производительность, гибкость и простоту использования, что позволяет реализовать все требуемые функции.

3.2 Разработка базы данных

Разработка базы данных веб-приложения для автоматизации работы библиотеки реализована с использованием объектно-реляционного отображения Flask-SQLAlchemy. Данный подход позволяет описывать структуру базы данных в виде классов на языке программирования Python, что обеспечивает наглядность кода и удобство взаимодействия с данными.

На основе спроектированной реляционной схемы были созданы следующие модели данных.

Модель «User» предназначена для хранения учетных записей пользователей системы. Python-код реализации таблицы представлен ниже:

```
class User(db.Model, UserMixin):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(255), unique=True,
nullable=False)
    password_hash = db.Column(db.String(255),
nullable=False)
    last_name = db.Column(db.String(100),
nullable=False)
    first_name = db.Column(db.String(100),
nullable=False)
    phone = db.Column(db.String(20))
    is_librarian = db.Column(db.Boolean,
default=False)
    is_active = db.Column(db.Boolean, default=True)
    registration_date = db.Column(db.DateTime,
default=datetime.utcnow)
```

Для работы с паролем в модели реализованы два метода. Метод `set_password` принимает пароль в открытом виде и сохраняет его хешированную версию с помощью функции `generate_password_hash`. Метод `check_password` сравнивает введенный пароль с хешем, используя функцию `check_password_hash`, и возвращает значение «True» при совпадении. Python-код реализации данных методов представлен ниже:

```
def set_password(self, password):
    self.password_hash = generate_password_hash
(password)
```

```
def check_password(self, password):
    return check_password_hash(self.password_hash,
password)
```

Модель «Book» хранит информацию о книгах, составляющих библиотечный фонд. Python-код реализации таблицы представлен ниже:

```
class Book(db.Model):
    __tablename__ = 'books'
    id = db.Column(db.Integer, primary_key=True)
    isbn = db.Column(db.String(17))
    title = db.Column(db.String(500),
nullable=False)
    description = db.Column(db.Text)
    publication_year = db.Column(db.Integer)
    total_copies = db.Column(db.Integer, default=0)
    available_copies = db.Column(db.Integer,
default=0)
    cover_url = db.Column(db.String(500))
    created_at = db.Column(db.DateTime,
default=datetime.utcnow)
```

Связи модели «Book» реализованы по типу «многие-ко-многим» с тремя справочными сущностями. Связь с моделью «Author» означает, что одна книга может иметь несколько авторов, и один автор может написать несколько книг. Связь с моделью «Genre» позволяет книге относиться к нескольким жанрам, а одному жанру – содержать множество книг. Связь с моделью «Publisher» отражает ситуацию, когда книга может быть издана несколькими издательствами, и одно издательство может выпускать множество книг.

Python-код реализации связей модели «Book» представлен ниже:

```
genres = db.relationship('Genre',
secondary='book_genres', backref='books')
```

```

        authors = db.relationship('Author',
secondary='book_authors', backref='books')
        publishers = db.relationship('Publisher',
secondary='book_publishers', backref='books')

```

Параметр `secondary` в каждом объявлении связи указывает на имя таблицы, которая хранит соответствия между книгами и соответствующими сущностями. Благодаря этому ORM автоматически управляет связями: при добавлении автора к книге с помощью метода `append` «SQLAlchemy» самостоятельно создаст необходимую запись в промежуточной таблице.

Модели «Author», «Genre» и «Publisher» являются справочными и хранят независимые сущности, используемые для классификации книг. Python-код реализации таблиц представлен ниже:

```

class Author(db.Model):
    __tablename__ = 'authors'
    id = db.Column(db.Integer, primary_key=True)
    last_name = db.Column(db.String(100),
nullable=False)
    first_name = db.Column(db.String(100),
nullable=False)
    middle_name = db.Column(db.String(100))
class Genre(db.Model):
    __tablename__ = 'genres'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), unique=True,
nullable=False)
class Publisher(db.Model):
    __tablename__ = 'publishers'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(255), unique=True,
nullable=False)

```


Модель «Author» содержит поля для фамилии, имени и отчества автора, причем поля `last_name` и `first_name` являются обязательными для заполнения. Модель «Genre» имеет поле `name`, которое хранит название жанра и должно быть уникальным. Модель «Publisher» аналогично содержит поле `name` – название издательства, также уникальное.

Python-код реализации таблиц `book_authors`, `book_genres`, `book_publishers` представлен ниже:

```
class BookAuthor(db.Model):
    __tablename__ = 'book_authors'
    book_id = db.Column(db.Integer,
db.ForeignKey('books.id'), primary_key=True)
    author_id = db.Column(db.Integer,
db.ForeignKey('authors.id'), primary_key=True)

class BookGenre(db.Model):
    __tablename__ = 'book_genres'
    book_id = db.Column(db.Integer,
db.ForeignKey('books.id'), primary_key=True)
    genre_id = db.Column(db.Integer,
db.ForeignKey('genres.id'), primary_key=True)

class BookPublisher(db.Model):
    __tablename__ = 'book_publishers'
    book_id = db.Column(db.Integer,
db.ForeignKey('books.id'), primary_key=True)
    publisher_id = db.Column(db.Integer,
db.ForeignKey('publishers.id'), primary_key=True)
```

Каждая из этих таблиц содержит ровно два поля, каждое из которых является внешним ключом на соответствующую таблицу. Оба поля вместе образуют составной первичный ключ, что предотвращает добавление дублирующихся пар связей. Например, в таблице `book_authors` нельзя дважды записать одну и ту же пару «книга – автор».

Модель «Reservation» отвечает за учет бронирования и выдачи книг читателям. Python-код реализации таблицы представлен ниже:

```
class Reservation(db.Model):
    __tablename__ = 'reservations'
    id = db.Column(db.Integer, primary_key=True)
    book_id = db.Column(db.Integer,
db.ForeignKey('books.id'), nullable=False)
    user_id = db.Column(db.Integer,
db.ForeignKey('users.id'), nullable=False)
    status = db.Column(db.String(20),
nullable=False)
    created_at = db.Column(db.DateTime,
default=datetime.utcnow)
    issued_at = db.Column(db.DateTime)
    due_date = db.Column(db.Date)
    returned_at = db.Column(db.DateTime)
    librarian_issued_id = db.Column(db.Integer,
db.ForeignKey('users.id'))
    librarian_returned_id = db.Column(db.Integer,
db.ForeignKey('users.id'))
```

В данной модели поле `status` определяет текущее состояние бронирования и может принимать три значения: «active» (книга забронирована, но еще не выдана), «issued» (книга выдана читателю) и «closed» (книга возвращена).

Python-код реализации связей в модели связей «Reservation» представлен ниже:

```
book_obj = db.relationship('Book',
backref='book_reservations_rel', lazy=True)
```

```

        user_rel = db.relationship('User',
foreign_keys=[user_id],
backref='user_reservations_rel', lazy=True)

        issued_by_rel = db.relationship('User',
foreign_keys=[librarian_issued_id],
backref='issued_by_reservations_rel', lazy=True)

        returned_by_rel = db.relationship('User',
foreign_keys=[librarian_returned_id],
backref='returned_by_reservations_rel', lazy=True)

        fines_rel = db.relationship('Fine',
backref='fine_reservation_rel', lazy=True)

```

Связь с моделью «Book» «многие-к-одному» означает, что много бронирований может ссылаться на одну книгу. Связь с моделью «User» реализована с помощью параметра `foreign_keys=[user_id]`, так как у модели есть несколько внешних ключей на таблицу `users`. Явное указание внешнего ключа необходимо, чтобы расширение SQLAlchemy правильно идентифицировала, какое именно поле используется для связи. Две отдельные связи с моделью «User» через поля `librarian_issued_id` и `librarian_returned_id` показывают, что один библиотекарь может оформить множество выдач и множество возвратов. Наконец, связь с моделью «Fine» позволяет одному бронированию иметь несколько штрафов, например, если читатель неоднократно нарушал сроки возврата.

Дополнительно в модели `Reservation` реализованы два свойства-вычислителя. Свойство `is_overdue` возвращает значение «True», если книга находится в статусе «выдана», и текущая дата превышает значение `due_date`. Свойство `days_overdue` возвращает количество дней просрочки, вычисляемое как разность между текущей датой и значением `due_date`.

Модель «Fine» предназначена для начисления и учета штрафов за просрочку возврата книг. Python-код реализации таблицы представлен ниже:

```
class Fine(db.Model):
    __tablename__ = 'fines'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer,
db.ForeignKey('users.id'), nullable=False)
    reservation_id = db.Column(db.Integer,
db.ForeignKey('reservations.id'), nullable=False)
    amount = db.Column(db.Numeric(10, 2),
nullable=False)
    reason = db.Column(db.String(255),
nullable=False)
    status = db.Column(db.String(20),
default='active')
    created_at = db.Column(db.DateTime,
default=datetime.utcnow)
    paid_at = db.Column(db.DateTime)
```

Поле `reason` содержит текстовое объяснение причины начисления, например, «Просрочка возврата на 5 дней». Поле `status` может принимать значения «active» (штраф не оплачен) или «paid» (штраф оплачен). Поле `created_at` автоматически фиксирует дату и время начисления штрафа с помощью метода `datetime.utcnow`.

В процессе разработки базы данных веб-приложения использовалась СУБД PostgreSQL. Все модели, описанные выше, после запуска приложения автоматически создаются в базе данных с помощью механизма миграций Flask-SQLAlchemy. При первом запуске приложения выполняются функции `create_all`, которая на основе объявленных классов-моделей генерирует соответствующие SQL-команды для создания таблиц, индексов и ограничений целостности.

В результате работы приложения в PostgreSQL формируется структура базы данных. Каждая таблица содержит первичные ключи, внешние ключи и ограничения, заданные в моделях.

Для визуального представления структуры базы данных в среде разработки была сгенерирована ERD-диаграмма на основе реального состояния базы данных PostgreSQL. ERD-диаграмма разработанной базы данных представлена в Приложении 1.

Данная диаграмма автоматически построена средствами системы управления базами данных и отображает все существующие таблицы, их столбцы с указанием типов данных, а также связи между таблицами с помощью первичных и внешних ключей.

3.3 Разработка функционала веб-приложения

Разработка пользовательского интерфейса и программной логики веб-приложения «КнигоЛов» велась согласованно в рамках клиент-серверной архитектуры. В данном разделе рассматриваются как визуальные компоненты интерфейса, так и ключевые программные модули, обеспечивающие функционирование системы.

Основой для всех страниц приложения служит шаблон «base.html». Он определяет общую структуру, включая настройки кодировки, адаптивной вёрстки, подключение внешних библиотек иконок и основного файла стилей, а также основной CSS-файл «style.css». Фрагмент HTML-кода, отвечающий за подключение стилей, представлен ниже:

```
<link rel="stylesheet" href="{{ url_for('static',  
filename='style.css') }}">  
  
<link rel="stylesheet"  
href="https://cdnjs.cloudflare.com/ajax/libs/font-  
awesome/6.4.0/css/all.min.css">
```

Использование функции `url_for('static', filename='...')` обеспечивает корректное формирование пути к статическим файлам независимо от конфигурации сервера, что является стандартной практикой во Flask-приложениях.

В данном шаблоне реализована верхняя навигационная панель «navbar». Реализация условного отображения элементов выполнена с использованием конструкции Jinja2 `{%if current_user.is_authenticated%`.

Переменная `current_user` предоставляется Flask-Login и содержит объект текущего авторизованного пользователя. Атрибут `is_librarian` определён в модели «User» и позволяет разграничивать доступ к административным разделам.

Фрагмент HTML-кода с реализацией разграничения видимости кнопок представлен в Приложении 2.

Также в шаблоне реализован контейнер для отображения временных сообщений системы: об успешном выполнении операций, ошибках или предупреждения. HTML-код реализации представлен ниже:

```
{% with messages =
get_flashed_messages(with_categories=true) %}
{% if messages %}
<div class="flash-messages">
{% for category, message in messages %}
<div class="flash flash-{{ category }}">
{{ message }}</div>
{% endfor %}</div>
{% endif %}
{% endwith %}
```

Реализация с использованием `get_flashed_messages (with_categories=true)` позволяет получать сообщения вместе с категорией, что даёт возможность применять различные стили оформления.

В веб-приложении требуется автоматически скрывать такие сообщения через 5 секунд после их появления. Данный функционал невозможно реализовать на серверном языке Python (Flask), так как отсчёт времени и удаление элементов должен происходить в браузере пользователя после того, как сервер уже отправил HTML-страницу и завершил соединение. Поэтому для решения этой задачи был использован минимальный JavaScript-код, который выполняется на стороне клиента и управляет отображением уведомлений без перезагрузки страницы. JavaScript-код реализации представлен ниже:

```
document.addEventListener('DOMContentLoaded',
function() {
    const flashMessages =
document.querySelectorAll('.flash');
    flashMessages.forEach(flash => {
        setTimeout(() => {
            flash.style.opacity = '0';
            setTimeout(() => flash.remove(), 300);
        }, 5000);});});
```

Адаптивное поведение интерфейса реализовано с использованием медиа-запросов в файле «style.css». Медиа-запросы позволяют изменять расположение элементов, размеры шрифтов и отступы в зависимости от ширины экрана устройства. Это необходимо для корректного отображения веб-приложения на мобильных устройствах, планшетах и настольных компьютерах. Минимальная поддерживаемая ширина экрана составляет 420 пикселей. Ниже представлен фрагмент CSS-кода, реализующий адаптивную трансформацию:

```
@media (max-width: 768px) {
    .nav-container {
        flex-direction: column;
        height: auto !important;
```

```
padding: 12px 15px !important;}
.nav-links {
  justify-content: center;
  width: 100%;
  gap: 8px;
  flex-wrap: wrap;}
.books-grid {
  grid-template-columns: repeat(auto-fill,
minmax(140px, 1fr));
  gap: 12px;}
.book-detail {
  grid-template-columns: 1fr;
  padding: 1rem;}}
```

Страница каталога книг в адаптивной верстке под экран телефона представлена на рисунке 15.

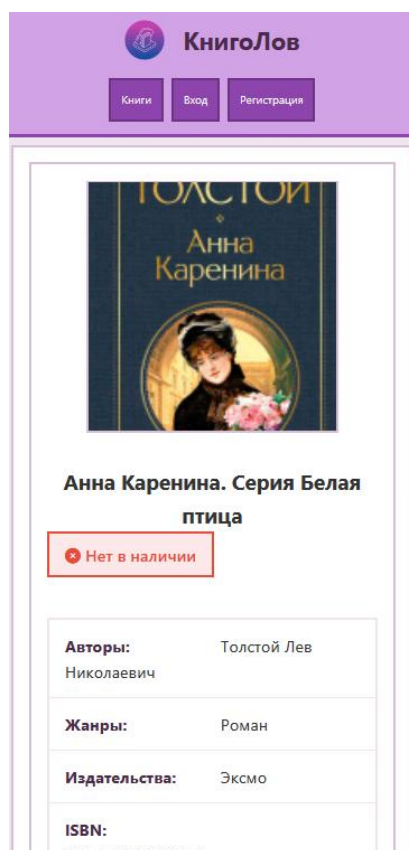


Рисунок 15 – Страница каталога книг в адаптивной верстке

Адаптивная вёрстка ограничена значением «768px», поскольку это стандартная точка перелома, разделяющая мобильные устройства и устройства с большим экраном. На экранах шире «768px» используется обычная версия интерфейса, которая не требует дополнительной адаптации, так как стандартные CSS-правила обеспечивают корректное отображение на планшетах и мониторах. При необходимости пороговое значение можно увеличить до «1024px» для включения планшетов в адаптивную версию.

Атрибут `flex-direction: column` преобразует горизонтальное расположение элементов навигационной панели в вертикальное, что особенно важно для узких экранов, где кнопки в одну строку не помещаются. Свойство `flex-wrap: wrap` позволяет кнопкам переноситься на следующую строку при недостатке места. Для страницы детального просмотра книги («book-detail») сетка из двух колонок (изображение и информация) заменяется на одну колонку, что делает содержимое удобным для чтения на маленьких экранах.

Эффект сжатия шапки при прокрутке страницы реализован с использованием JavaScript-обработчика события `scroll`. Данный функционал не может быть реализован на серверной стороне, так как отслеживание положения прокрутки происходит исключительно в браузере пользователя после полной загрузки страницы.

Все остальные шаблоны наследуют структуру от «base.html», переопределяя только блок `content` с помощью конструкции `{% block content %}{% endblock %}`. Это обеспечивает единообразие интерфейса, централизованное управление общими элементами и упрощает поддержку проекта при внесении изменений.

Регистрация пользователей реализована в маршруте `/register`. Данный маршрут обрабатывает как GET-запросы – отображение формы регистрации, так и POST-запросы – отправка заполненной формы. При получении POST-запроса из формы извлекаются следующие данные: адрес электронной почты, пароль, имя, фамилия и номер телефона. Все поля, кроме

телефона, являются обязательными для заполнения. Страница регистрации пользователя представлена на рисунке 16.

The screenshot shows a web application interface for 'КнигоЛов' (BookLover). The header is blue and contains the site logo, the name 'КнигоЛов', and three navigation buttons: 'Книги' (Books), 'Вход' (Login), and 'Регистрация' (Registration). The main content area is white and features a central registration form titled 'Регистрация' with a user icon. The form includes input fields for 'Фамилия' (Surname) and 'Имя' (Name), an 'Email' field, a 'Телефон' (Phone) field, and a 'Пароль' (Password) field. Below the password field, there is a note 'Минимум 6 символов' (Minimum 6 characters). A blue button labeled 'Зарегистрироваться' (Register) is positioned below the form. At the bottom of the form, there is a link 'Уже есть аккаунт? Войти' (Already have an account? Login). The footer is blue and contains the copyright notice '© 2026 КнигоЛов - Библиотечная система' on the left and 'Продукт в разработке' (Product in development) on the right.

Рисунок 16 – Страница регистрации пользователя

Первым этапом обработки выполняется проверка на существование пользователя с указанным адресом электронной почты. Это необходимо для предотвращения дублирования учётных записей, так как адрес электронной почты используется в качестве уникального идентификатора для входа в систему. Проверка выполняется с помощью метода `filter_by(email=email).first()`.

Если пользователь с таким `email` не найден, создаётся новый объект модели «User». Пароль не сохраняется в открытом виде – вместо этого вызывается метод `set_password(password)`, который с помощью функции `generate_password_hash` из модуля «werkzeug.security» создаёт хешированную версию пароля. Хеширование является критически важным для безопасности системы.

После успешной регистрации пользователю отображается сообщение об успешной регистрации, и выполняется перенаправление на страницу входа.

Фрагмент Python-кода реализации регистрации представлен в Приложении 3.

Авторизация пользователей реализована в маршруте `/login`. Страница авторизации пользователя представлена на рисунке 17.

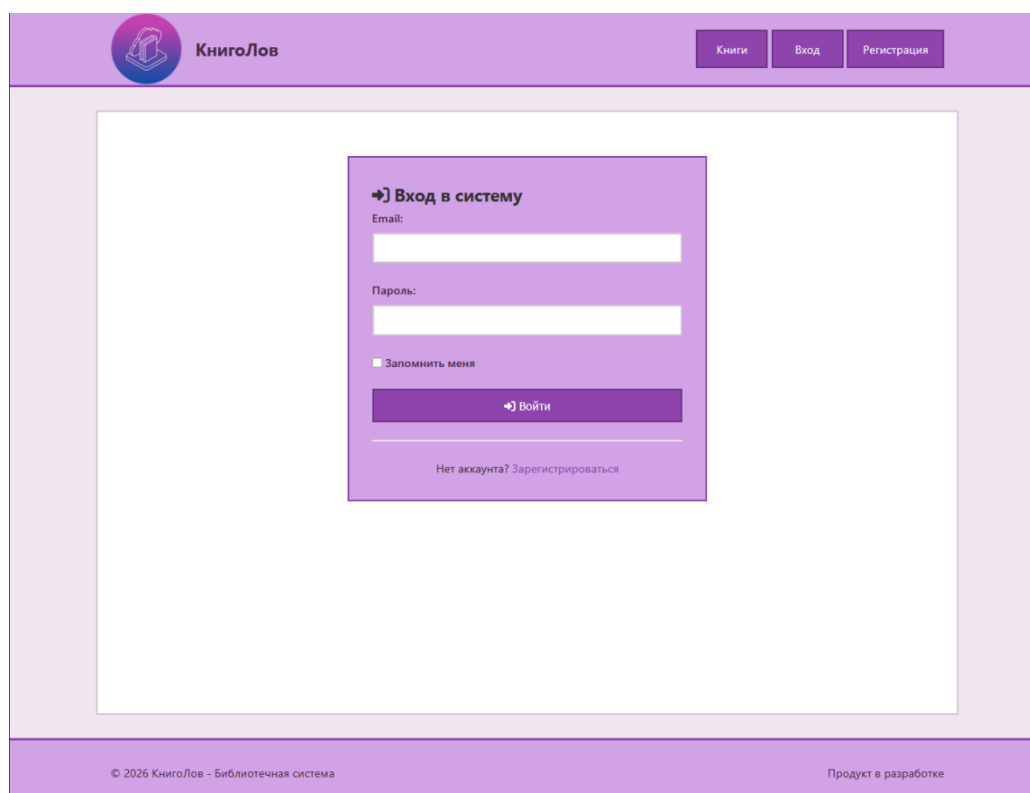
The image shows a web application interface for 'КнигоЛов' (BookLover). The header is purple with a logo on the left and navigation links 'Книги', 'Вход', and 'Регистрация' on the right. The main content area is white and contains a purple login form. The form has the title 'Вход в систему' with a key icon, followed by 'Email:' and a text input field, 'Пароль:' and another text input field, a checkbox labeled 'Запомнить меня', and a purple button labeled 'Войти' with a key icon. Below the button is a link 'Нет аккаунта? Зарегистрироваться'. The footer is purple and contains the copyright notice '© 2026 КнигоЛов - Библиотечная система' on the left and 'Продукт в разработке' on the right.

Рисунок 17 – Страница авторизации пользователя

При POST-запросе система извлекает `email` и пароль из формы, выполняет поиск пользователя в базе данных. При успешной аутентификации вызывается функция `login_user(user, remember=remember)`. Параметр `remember` управляет созданием постоянной сессии. После входа обычные читатели перенаправляются на главную страницу, библиотекари – на панель управления. Python-код реализации авторизации представлен ниже:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
```

```

return redirect(url_for('admin_panel' if
current_user.is_librarian else 'index'))
if request.method == 'POST':
email = request.form.get('email')
password = request.form.get('password')
remember = True if request.form.get('remember') else
False
user = User.query.filter_by(email=email).first()
if user and user.check_password(password) and
user.is_active:
login_user(user, remember=remember)
logger.info(f"User {email} logged in successfully")
return redirect(url_for('admin_panel' if
user.is_librarian else 'index'))
else:
flash('Неверный email, пароль 'danger')
return render_template('login.html')

```

Выход из аккаунта реализован в маршруте `/logout`. Функция `logout_user()` завершает сессию. Python-код реализации выхода из аккаунта представлен ниже:

```

@app.route('/logout')
@login_required
def logout():
    logout_user()
    flash('Вы успешно вышли из системы', 'success')
    return redirect(url_for('index'))

```

Управление книжным фондом реализовано в трёх маршрутах: `/books/add` (добавление книги), `/books/<int:book_id>/edit` (редактирование) и `/books/<int:book_id>/delete` (удаление). Доступ к этим маршрутам ограничен только для пользователей с ролью библиотекаря.

Добавление книги обрабатывается маршрутом `/books/add`. При GET-запросе отображается форма, содержащая поля для ввода названия, ISBN, года издания, количества экземпляров, ссылки на обложку и описания. Также в форму передаются списки существующих авторов, жанров и издательств для выбора через множественные отметки. Страница добавления новой книги представлена на рисунке 18.

+ Добавить новую книгу [← Назад](#)

Основная информация

Название книги *

ISBN

Год издания

Всего экземпляров *

Доступно экземпляров *

Ссылка на обложку

Описание

Авторы

Выберите авторов:

☐ Толстой Лев Николаевич	☐ Достоевский Федор Михайлович	☐ Пушкин Александр Сергеевич
☐ Гоголь Николай Васильевич	☐ Чехов Антон Павлович	☐ Булгаков Михаил Афанасьевич

Рисунок 18 – Страница добавления новой книги

При POST-запросе выполняется сложная логика создания книги, включающая возможность добавления новых авторов, жанров и издательств непосредственно в процессе заполнения формы. Это реализовано с помощью текстовых полей `new_authors`, `new_genres` и `new_publishers`, в которые библиотекарь может ввести данные построчно.

Обработка новых авторов выполняется следующим образом: текст из поля `new_authors` разбивается по символу переноса строки, каждая строка очищается от пробелов. Если строка не пуста, она разбивается на части по пробелам. Первая часть интерпретируется как фамилия, вторая – как имя,

оставшиеся части объединяются в отчество (если есть). После этого выполняется проверка на существование автора с такими же фамилией и именем. Если автор не найден, создаётся новый объект `Author`, добавляется в сессию базы данных, и вызывается `db.session.flush()` для немедленного получения присвоенного идентификатора. Использование `flush()` вместо `commit()` необходимо потому, что автор должен быть сохранён в базу данных для получения идентификатора, но основной `commit()` будет выполнен позже после создания всех связанных сущностей.

Аналогичным образом обрабатываются новые жанры и издательства. Для них проверка уникальности выполняется по полю `name`.

Python-код обработки новых авторов представлен ниже:

```
new_authors_raw = request.form.get('new_authors',
    '').strip()
if new_authors_raw:
    for author_line in new_authors_raw.split('\n'):
        author_line = author_line.strip()
        if not author_line:
            continue
        parts = author_line.split()
        if len(parts) >= 2:
            last_name = parts[0]
            first_name = parts[1]
            middle_name = ' '.join(parts[2:]) if len(parts) > 2
            else None
            existing = Author.query.filter_by(last_name=last_name,
                first_name=first_name).first()
            if not existing:
                new_author = Author(last_name=last_name,
                    first_name=first_name, middle_name=middle_name)
                db.session.add(new_author)
```

```
db.session.flush()
```

После создания всех новых сущностей создаётся объект «Book». Для этого из формы извлекаются значения всех полей и преобразовываются.

Логика редактирования аналогична добавлению, но с одним отличием: перед обновлением связей списки `book.authors`, `book.genres` и `book.publishers` очищаются, после чего заполняются заново на основе данных из формы. Это необходимо для корректной обработки ситуации, когда из книги удаляется какой-либо автор, жанр или издательство. Страница редактирования книги представлена на рисунке 19.

Редактировать книгу [← Назад](#)

Основная информация

Название книги *

Война и мир

ISBN

978-5-17-090630-3

Год издания

1869

Всего экземпляров *

5

Доступно экземпляров *

3

Ссылка на обложку

https://cdn.azbooka.ru/cv/w1100/4d7ac101-3774-48c9-bf46-5067276cf101.jpg

Описание

Роман-эпопея Льва Толстого

Авторы

Рисунок 19 – Страница редактирования книги

Удаление книги реализовано в маршруте `/books/<int:book_id>/delete`. Перед удалением выполняется критически важная проверка: наличие активных бронирований для данной книги. Если такие бронирования существуют, удаление блокируется, и пользователю выводится сообщение об ошибке. Это необходимо для обеспечения целостности данных: если бы книга с активными бронированиями была удалена, ссылки на неё в таблице «reservations» стали бы некорректными.

Выдача и возврат книг реализованы в маршрутах `/reservations/<int:reservation_id>/issue` и `/reservations/<int:reservation_id>/return` соответственно, код которых представлен в Приложении 4. Доступ к обоим маршрутам разрешен только библиотекарям.

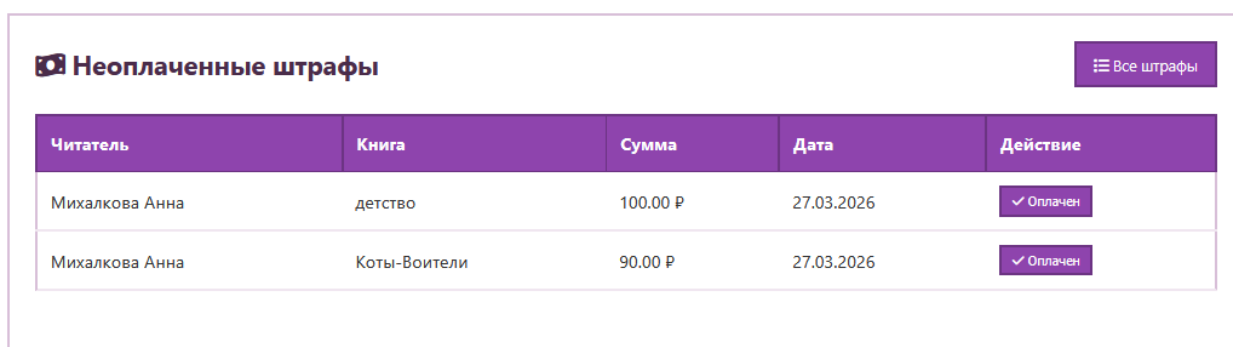
Выдача книги выполняется из бронирования со статусом `active`. При выдаче сначала проверяется наличие неоплаченных штрафов у читателя с помощью функции `get_user_active_fine_amount`. Если сумма штрафов больше нуля, выдача блокируется – это стимулирует читателей своевременно оплачивать задолженности. При успешной выдаче статус бронирования изменяется на `issued`, фиксируются дата и время выдачи с помощью `datetime.utcnow()`, устанавливается планируемая дата возврата – через 14 дней от текущей даты, а также сохраняется идентификатор библиотекаря, оформившего выдачу.

Возврат книги выполняется из бронирования со статусом `issued`. При возврате статус изменяется на `closed`, фиксируются дата и время возврата, идентификатор библиотекаря, принявшего возврат, а количество доступных экземпляров книги увеличивается на единицу.

Ключевая логика при возврате – начисление штрафа при просрочке. Если поле `due_date` меньше текущей даты, значит читатель вернул книгу позже установленного срока. Количество дней просрочки вычисляется как разность между текущей датой и `due_date`, после чего штраф начисляется из расчёта 10 рублей за каждый день. Создаётся объект модели «`Fine`», который связывается с пользователем и бронированием, содержит сумму штрафа и текстовое описание причины.

Важно отметить, что штраф начисляется только при фактическом возврате, что позволяет корректно обрабатывать ситуации, когда читатель вернул книгу с просрочкой, но до этого штраф не начислялся.

Автоматическое начисление штрафов реализовано в функции `check_overdue_reservations`, Python-код которой представлен в Приложении 5. Функция предназначена для периодического запуска через планировщик задач и не привязана к конкретному HTTP-запросу. Она находит все бронирования со статусом `issued`, у которых `due_date` меньше текущей даты. Для каждого проверяется отсутствие активного штрафа – если штраф ещё не начислен, он создаётся. Это предотвращает дублирование начислений при многократном запуске функции. Отображение штрафов на странице панели управления у библиотекаря представлена на рисунке 20.



Читатель	Книга	Сумма	Дата	Действие
Михалкова Анна	детство	100.00 Р	27.03.2026	✓ Оплачен
Михалкова Анна	Коты-Воители	90.00 Р	27.03.2026	✓ Оплачен

Рисунок 20 – Отображение штрафов

Поиск и фильтрация книг реализованы в маршруте `/books`, полный код которого представлен в Приложении 6. Параметры фильтрации передаются через строку запроса GET-методом, что позволяет сохранять параметры в URL и делиться ссылками на отфильтрованные результаты.

Доступны следующие параметры фильтрации: `search` – поиск по названию книги с использованием регистронезависимого сопоставления, `author_id` – фильтрация по конкретному автору, `genre_id` – фильтрация по жанру, `available_only` – отображение только книг, имеющих в наличии хотя бы в одном экземпляре.

Итоговый набор книг сортируется по названию и извлекается из базы данных. Также отдельными запросами получают списки всех авторов и жанров для заполнения выпадающих списков в форме фильтрации.

Страница каталога и фильтрации книг реализована в шаблоне «books_list.html», HTML-код которого представлен в Приложении 7. Страница содержит форму фильтрации, расположенную в верхней части, и сетку карточек книг, занимающую основную область. Страница с каталогом книг представлена на рисунке 21.

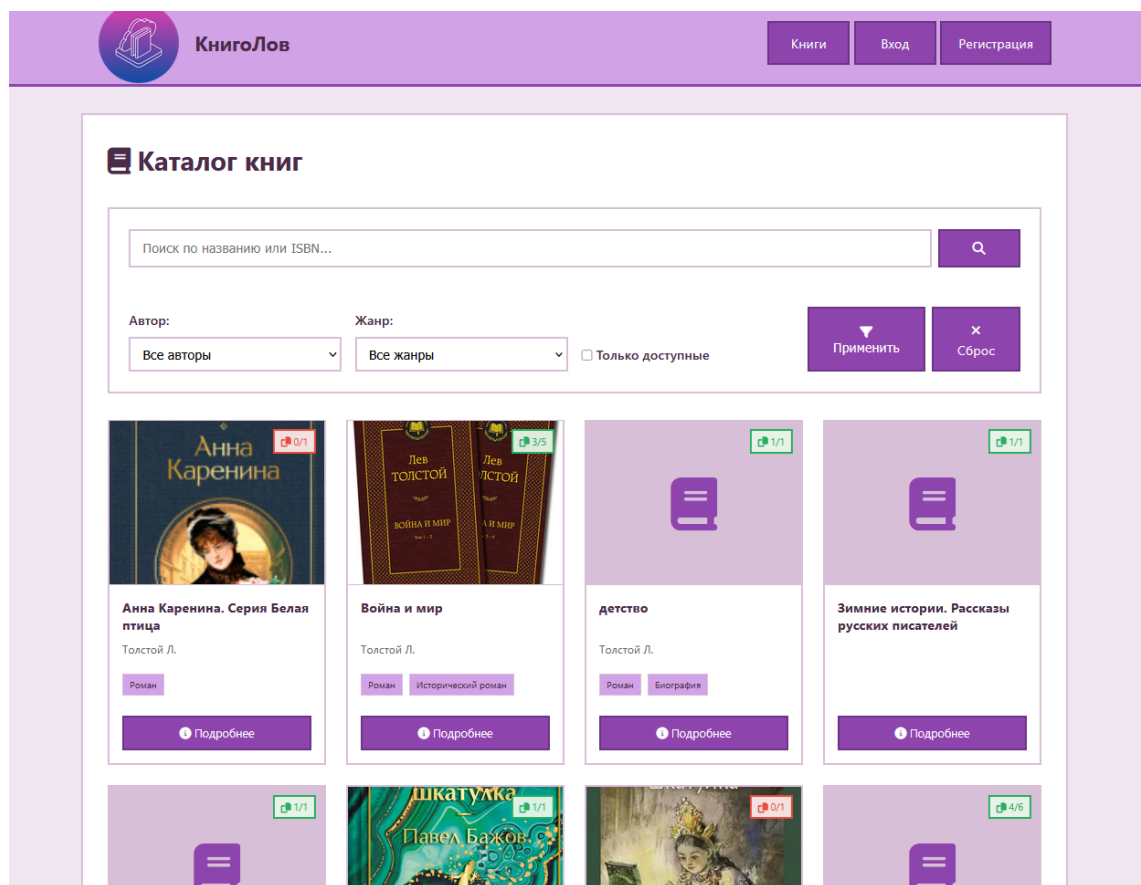


Рисунок 21 – Страница с каталогом книг

Форма фильтрации использует метод GET, значения текущих фильтров подставляются обратно в поля формы с помощью `request.args.get()`. Для поля поиска значение подставляется в атрибут `value` текстового поля. Для выпадающих списков авторов и жанров используется условная проверка: если значение параметра совпадает с идентификатором элемента, добавляется атрибут `selected`. Фильтр «Только доступные» представлен отметкой, у которого при наличии параметра `available_only` добавляется атрибут `checked`. Кнопка «Сброс» ведёт на тот же маршрут без параметров, что очищает все фильтры.

Каждая карточка книги содержит обложку с угловым индикатором количества доступных экземпляров, название, список авторов в сокращённом формате – фамилия и инициал имени, до трёх жанров, а также кнопки «Подробнее» для перехода к детальной информации о книге. Для авторизованного библиотекаря дополнительно отображается кнопка «Редактировать», ведущая на страницу редактирования книги.

Угловой индикатор количества экземпляров имеет цветовую разграничение: зелёный фон с текстом «доступно» применяется, если `available_copies > 0`, красный фон с текстом «нет в наличии» – если экземпляры отсутствуют. Это достигается условным добавлением CSS-классов `available` или `unavailable` к элементу `span`.

При отсутствии книг, соответствующих критериям фильтрации, вместо сетки карточек отображается блок с иконкой, заголовком «Книги не найдены» и предложением изменить параметры поиска.

Авторизованный читатель может забронировать книгу, если она имеется в наличии хотя бы в одном экземпляре. Система проверяет лимит активных бронирований – не более пяти книг на одного читателя. При бронировании количество доступных экземпляров автоматически уменьшается. Библиотекарям бронирование книг запрещено.

Отображение в личном кабинете читателя текущих бронирований, выданных книг и штрафов реализовано в маршруте `/profile` и шаблоне «`profile.html`». Четыре запроса к базе данных извлекают бронирования с различными статусами: `active` – для активных бронирований, `issued` – для выданных книг, `closed` – для истории. Сумма активных штрафов вычисляется в шаблоне с использованием цикла `for`. Python-код реализации маршрута представлен в Приложении 8, HTML-код шаблона – в Приложении 9. Страница личного кабинета представлена на рисунке 22.

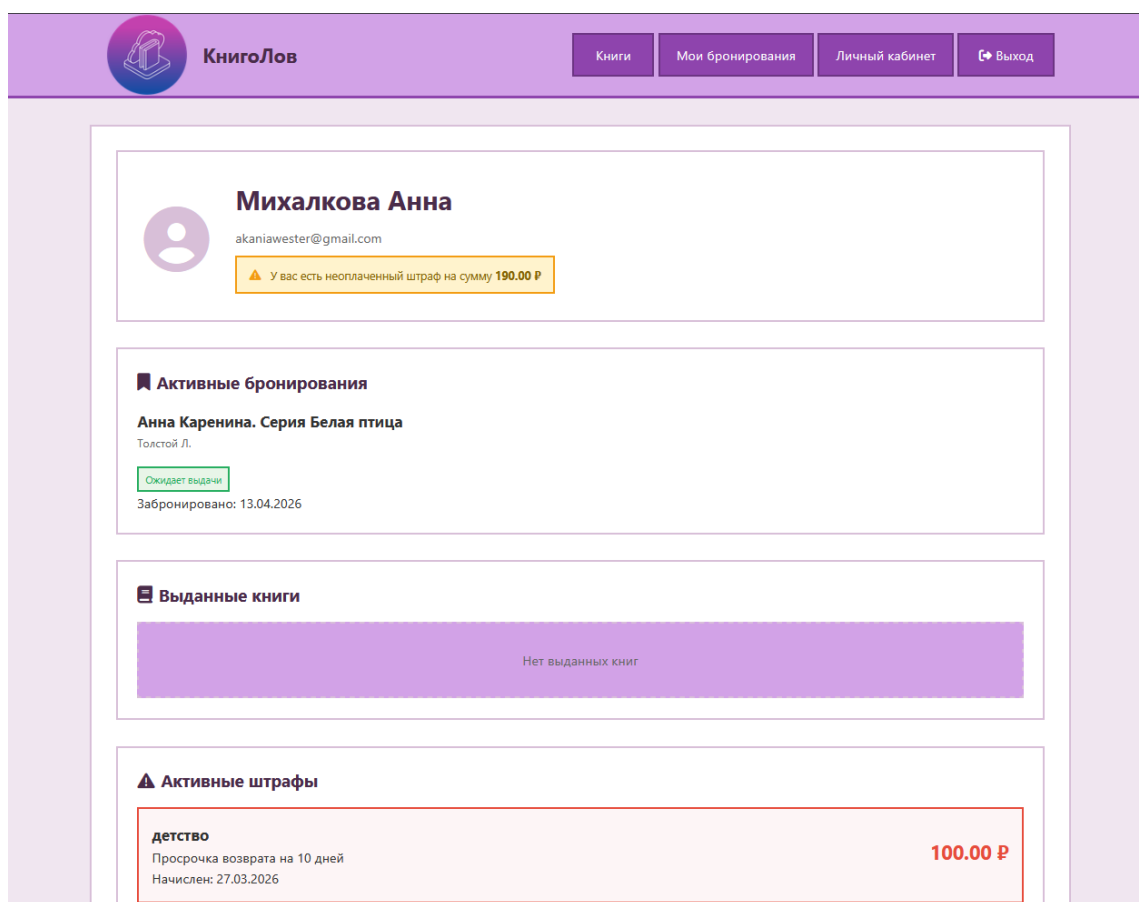


Рисунок 22 – Страница личного кабинета читателя

Помимо основных требований индивидуального задания, в разработанном веб-приложении «КнигоЛов» представлены следующие возможности:

- панель управления библиотекаря со сводной статистикой, списком последних бронирований, неоплаченных штрафов и просроченных возвратов книг ;
- просмотр данных о штрафах для библиотекаря с поиском по фамилии читателя;
- просмотр данных о бронированиях с фильтрацией по статусу для библиотекаря;
- отображение уведомлений с автоматическим скрыванием через 5 секунд;
- эффект сжатия шапки при прокрутке страницы;
- страницы ошибок 404 и 500;

- обработка битых ссылок на изображения обложек книг;
- логирование действий пользователей и ошибок системы;
- блокировка выдачи книги при наличии неоплаченных штрафов;
- ограничение на количество одновременных бронирований (не более пяти книг);
- защита от удаления книги с активными бронированиями;
- отображение суммы активных штрафов в личном кабинете читателя.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была достигнута поставленная цель – разработано веб-приложение «КнигоЛов», предназначенное для автоматизации процессов управления библиотекой, обеспечивающее учёт книжного фонда, бронирование и выдачу литературы, контроль задолженностей и взаимодействие с читателями через веб-интерфейс.

На первом этапе работы был проведён анализ предметной области, в ходе которого были выявлены особенности функционирования библиотек различных типов, а также основные проблемы, связанные с отсутствием автоматизации, такие как ручной учёт книг и читателей, сложность контроля сроков возврата и ограниченный доступ читателей к информации о наличии книг. Дополнительно был выполнен анализ существующих программных решений, таких как «ИРБИС» и «1С:Библиотека», что позволило определить их недостатки, включая сложность освоения, высокую стоимость внедрения и отсутствие полноценного личного кабинета для читателей, и обосновать необходимость разработки собственного веб-приложения.

Также были сформулированы функциональные и нефункциональные требования к разрабатываемой системе. Определение требований позволило установить перечень функций, необходимых для читателей и библиотекарей, а также задать ограничения, обеспечивающие надёжность и удобство работы с системой, включая требования к адаптивности интерфейса, безопасности хранения паролей и совместимости с различными браузерами.

На этапе проектирования была разработана архитектура веб-приложения, основанная на клиент-серверной модели с использованием объектно-реляционного отображения для взаимодействия с базой данных. Кроме того, была спроектирована структура базы данных, а также разработан пользовательский интерфейс в виде вайрфреймов, учитывающий особенности взаимодействия различных категорий пользователей – гостей, читателей и библиотекарей.

В процессе разработки были выбраны инструментальные средства, обеспечивающие реализацию проекта. На этапе реализации была создана база данных в соответствии с разработанной моделью, а также реализован функционал веб-приложения.

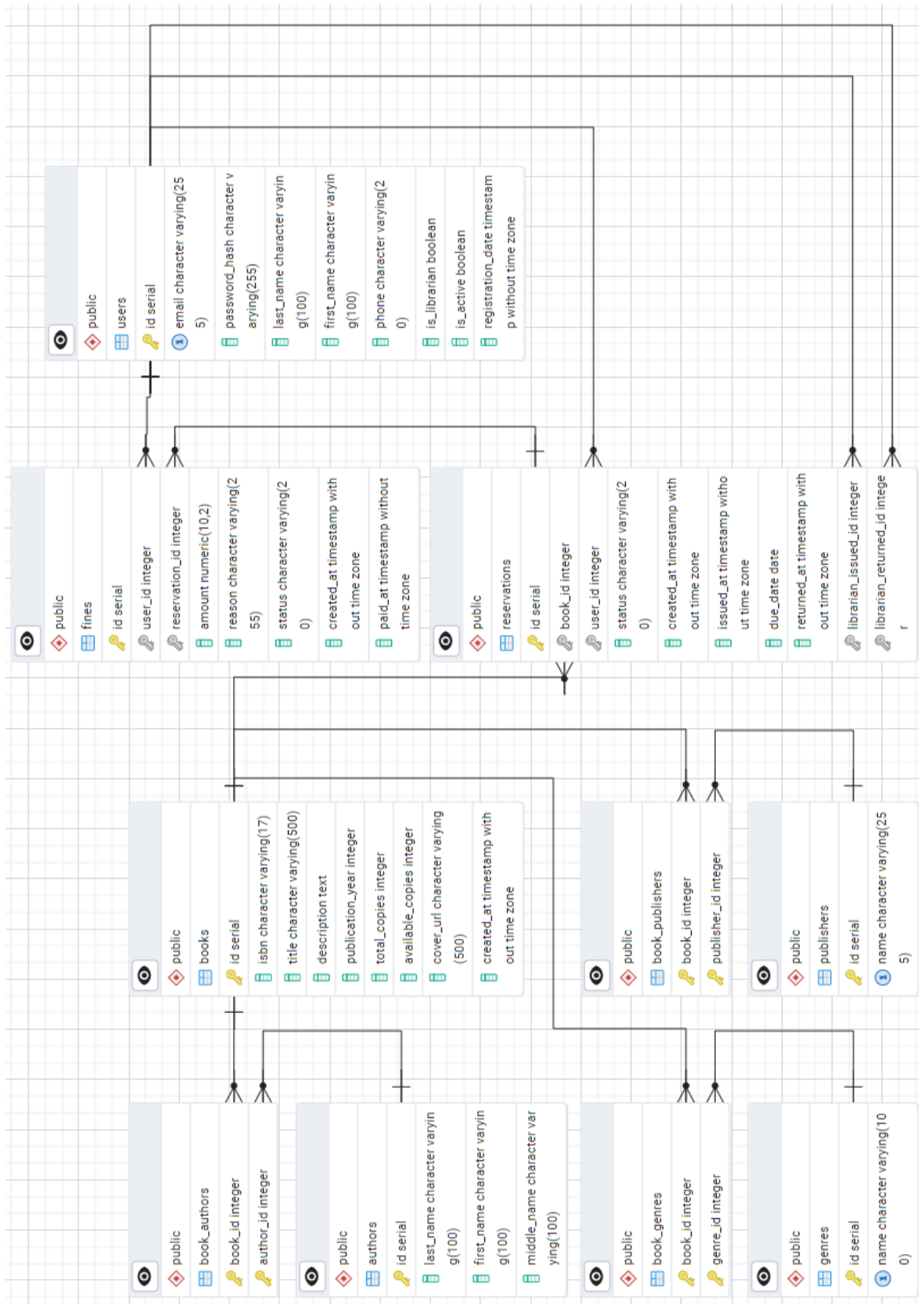
Разработанное веб-приложение может быть использовано в деятельности городских библиотек, библиотек учебных заведений и культурных организаций для повышения эффективности управления книжным фондом, снижения временных затрат сотрудников и улучшения качества обслуживания читателей.

Таким образом, все поставленные задачи дипломного проекта были успешно решены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Винокуров И.В. Разработка веб-приложений с использованием фреймворка Flask. Учебное пособие для вузов. – Санкт-Петербург: Лань, 2025. – 64 с.
2. Гаврилов, А. В. Проектирование реляционных баз данных : учебное пособие. – Москва : КноРус, 2025. – 231 с.
3. Доусон, М. Програмируем на Python : перевод с английского – 3-е изд. – Санкт-Петербург : Питер, 2023. – 416 с.
4. 3. Рахматуллаев, М. А. Проектирование информационно-библиотечных систем : учебник. – Москва : Инфра-М, 2025. – 286 с.
5. Рогов, Е.В. PostgreSQL 16 изнутри; Postgres Professional. – Москва : ДМК Пресс, 2024. – 664 с.
6. Функциональные и нефункциональные требования [Электронный ресурс] // Системный анализ: Простыми словами – URL: <https://setka.ru/posts/0191f93f-e8de-4638-b219-1a307ec39c> (дата обращения: 16.03.2026).
7. Описание логической структуры базы данных [Электронный ресурс] // ДБ-СЕРВИС – URL: <https://www.dbserv.ru/blog/logicheskaya-struktura-bazy-dannyh> (дата обращения: 23.03.2026).
8. Что значит объектно-реляционное отображение (ORM) и зачем это нужно [Электронный ресурс] // Макхост – URL: <https://mchost.ru/articles/chto-znachit-obektno-relyaczionnoe-otobrazhenie-orm-i-zachem-eto-nuzhno/> (дата обращения: 24.03.2026).
9. Онлайн-сервис для создания диаграмм и схем «draw.io» [Электронный ресурс] – URL: <https://app.diagrams.net/> (дата обращения: 29.03.2026).
10. Документация к PostgreSQL 18.3 [Электронный ресурс] – URL: <https://postgrespro.ru/docs/postgresql/current> (дата обращения: 03.04.2026).

ERD-диаграмма разработанной базы данных



HTML-код разграничения видимости кнопок

```
<div class="nav-links">
<a href="{{ url_for('books_list') }}">Книги</a>
{% if current_user.is_authenticated %}
{% if current_user.is_librarian %}
<a href="{{ url_for('admin_panel') }}">Панель
управления</a>
<a href="{{ url_for('reservations_list')
}}">Бронирования</a>
<a href="{{ url_for('admin_fines') }}">Штрафы</a>
{% else %}
<a href="{{ url_for('reservations_list') }}">Мои
бронирования</a>
<a href="{{ url_for('profile') }}">Личный кабинет</a>
{% endif %}
<a href="{{ url_for('logout') }}" class="logout-
btn">Выход</a>
{% else %}
<a href="{{ url_for('login') }}">Вход</a>
<a href="{{ url_for('register') }}">Регистрация</a>
{% endif %}
</div>
```

Python-код реализации регистрации пользователя

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('index'))
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')
        first_name = request.form.get('first_name')
        last_name = request.form.get('last_name')
        phone = request.form.get('phone')
        user = User.query.filter_by(email=email).first()
        if user:
            flash('Пользователь с таким email уже существует',
                  'danger')
            return redirect(url_for('register'))
        new_user = User(email=email, last_name=last_name,
                        first_name=first_name, phone=phone, is_librarian=False)
        new_user.set_password(password)
        db.session.add(new_user)
        db.session.commit()
        flash('Регистрация успешна! Теперь вы можете войти в
систему.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html')
```

Python-код реализации выдачи и возврата книг

```

@app.route('/reservations/<int:reservation_id>/issue',
methods=['POST'])
@login_required
def issue_book(reservation_id):
    if not current_user.is_librarian:
        flash('Доступ запрещен', 'danger')
        return redirect(url_for('index'))
    reservation =
Reservation.query.get_or_404(reservation_id)
    if reservation.status != 'active':
        flash('Нельзя оформить выдачу для этого бронирования',
'danger')
    return redirect(url_for('reservations_list'))
    fine_amount =
get_user_active_fine_amount(reservation.user_id)
    if fine_amount > 0:
        flash(f'НЕЛЬЗЯ ВЫДАТЬ КНИГУ! У читателя
{reservation.user_rel.last_name}
{reservation.user_rel.first_name} есть неоплаченный
штраф на сумму {fine_amount} Р. Сначала оплатите
штраф.', 'danger')
    return redirect(url_for('reservations_list'))
    try:
        reservation.status = 'issued'
        reservation.issued_at = datetime.utcnow()
        reservation.due_date = date.today() +
timedelta(days=0.1)

```

```
reservation.librarian_issued_id = current_user.id
db.session.commit()
flash('Выдача книги оформлена', 'success')
except Exception as e:
db.session.rollback()
logger.error(f"Error issuing book: {str(e)}")
flash('Ошибка при оформлении выдачи', 'danger')
return redirect(url_for('reservations_list'))
@app.route('/reservations/<int:reservation_id>/return',
methods=['POST'])
@login_required
def return_book(reservation_id):
if not current_user.is_librarian:
flash('Доступ запрещен', 'danger')
return redirect(url_for('index'))
reservation =
Reservation.query.get_or_404(reservation_id)
book = Book.query.get(reservation.book_id)
if reservation.status != 'issued':
flash('Нельзя оформить возврат для этого бронирования',
'danger')
return redirect(url_for('reservations_list'))
try:
reservation.status = 'closed'
reservation.returned_at = datetime.utcnow()
reservation.librarian_returned_id = current_user.id
if book:
book.available_copies += 1
if reservation.due_date and reservation.due_date <
date.today():
```

```
overdue_days = (date.today() -
reservation.due_date).days
fine_amount = overdue_days * 10.00
fine = Fine(user_id=reservation.user_id,
reservation_id=reservation.id, amount=fine_amount,
reason=f'Просрочка возврата на {overdue_days} дней')
db.session.add(fine)
db.session.commit()
flash('Возврат книги оформлен', 'success')
except Exception as e:
db.session.rollback()
logger.error(f"Error returning book: {str(e)}")
flash('Ошибка при оформлении возврата', 'danger')
return redirect(url_for('reservations_list'))
```

Python-код реализации автоматического начисления штрафов

```
def check_overdue_reservations():
    overdue_reservations = Reservation.query.filter(
        Reservation.status == 'issued',
        Reservation.due_date < date.today()).all()
    for reservation in overdue_reservations:
        existing_fine = Fine.query.filter_by(
            reservation_id=reservation.id,
            status='active').first()
        if not existing_fine:
            overdue_days = (date.today() -
                reservation.due_date).days
            fine_amount = overdue_days * 10.00
            fine = Fine(
                user_id=reservation.user_id,
                reservation_id=reservation.id,
                amount=fine_amount,
                reason=f'Просрочка возврата на {overdue_days} дней')
            db.session.add(fine)
        try:
            db.session.commit()
            logger.info(f"Checked overdue reservations:
                {len(overdue_reservations)} found")
        except Exception as e:
            db.session.rollback()
            logger.error(f"Error checking overdue reservations:
                {str(e)}")
```

Python-код реализации поиска и фильтрации книг

```
@app.route('/books')
def books_list():
    query = Book.query
    search = request.args.get('search')
    if search:
        query = query.filter(Book.title.ilike(f'%{search}%'))
    author_id = request.args.get('author_id')
    if author_id:
        query = query.join(Book.authors).filter(Author.id ==
        author_id)
    genre_id = request.args.get('genre_id')
    if genre_id:
        query = query.join(Book.genres).filter(Genre.id ==
        genre_id)
    available_only = request.args.get('available_only')
    if available_only:
        query = query.filter(Book.available_copies > 0)
    books = query.order_by(Book.title).all()
    genres = Genre.query.all()
    authors = Author.query.all()
    return render_template('books_list.html',
    books=books,
    genres=genres,
    authors=authors)
```


HTML-код реализации страницы с каталогом и фильтрацией книг

```
{% extends "base.html" %}
{% block title %}Книги - КнигоЛов {%}
{% endblock %}
{% block content %}
<div class="page-header">
<h1><i class="fas fa-book"></i> Каталог книг</h1>
{% if current_user.is_librarian %}
<a href="{{ url_for('book_add') }}" class="btn">
<i class="fas fa-plus"></i> Добавить книгу
</a>{% endif %}</div>
<div class="search-filters">
<form method="GET" class="filter-form">
<div class="search-box">
<input type="text"
name="search"
placeholder="Поиск по названию или ISBN..."
value="{{ request.args.get('search', '') }}"
class="form-control">
<button type="submit" class="btn">
<i class="fas fa-search"></i>
</button></div>
<div class="filter-grid">
<div class="filter-group">
<label>Автор:</label>
<select name="author_id" class="form-control">
<option value="">Все авторы</option>
{% for author in authors %}
```

```
<option value="{{ author.id }}"
{% if request.args.get('author_id')|int == author.id
%}selected{% endif %}>
{{ author.last_name }} {{ author.first_name }}
</option>
{% endfor %}
</select></div>
<div class="filter-group">
<label>Жанр:</label>
<select name="genre_id" class="form-control">
<option value="">Все жанры</option>
{% for genre in genres %}
<option value="{{ genre.id }}"
{% if request.args.get('genre_id')|int == genre.id
%}selected{% endif %}>
{{ genre.name }}
</option>{% endfor %}
</select></div>
<div class="filter-group">
<label class="checkbox-label">
<input type="checkbox"
name="available_only"
value="1"
{% if request.args.get('available_only') %}checked{%
endif %}>
Только доступные
</label>
</div>
<div class="filter-actions">
<button type="submit" class="btn">
```

```

<i class="fas fa-filter"></i> Применить
</button>
<a href="{{ url_for('books_list') }}" class="btn">
<i class="fas fa-times"></i> Сброс
</a></div></div>
</form></div>
{% if books %}
<div class="books-grid">
{% for book in books %}
<div class="book-card">
<div class="book-cover-wrapper">
<div class="book-cover">
{% if book.cover_url %}

{% else %}
<div class="no-cover">
<i class="fas fa-book"></i>
</div>{% endif %}</div>
<div class="book-status">
<span class="copies {% if book.available_copies > 0
%}available{% else %}unavailable{% endif %}">
<i class="fas fa-copy"></i> {{ book.available_copies
}}/{{ book.total_copies }}
</span></div></div>
<div class="book-info">
<h3>{{ book.title }}</h3>
<p class="book-authors">
{% for author in book.authors %}
{{ author.last_name }} {{ author.first_name[0] }}. {% if
not loop.last %}, {% endif %}

```

```

{% endfor %}</p>
<div class="book-genres">
{% for genre in book.genres[:3] %}
<span class="genre-tag">{{ genre.name }}</span>
{% endfor %}</div>
<div class="book-actions">
<a href="{{ url_for('book_detail', book_id=book.id) }}"
class="btn-small">
<i class="fas fa-info-circle"></i> Подробнее</a>
{% if current_user.is_librarian %}
<a href="{{ url_for('book_edit', book_id=book.id) }}"
class="btn-small">
<i class="fas fa-edit"></i> Редактировать
</a>{% endif %}
</div></div>
</div>{% endfor %}
</div>{% else %}
<div class="empty-state">
<i class="fas fa-book fa-3x"></i>
<h3>Книги не найдены</h3>
<p>Попробуйте изменить параметры поиска</p>
</div>{% endif %}{% endblock %}

```

Python-код реализации личного кабинета читателя

```
@app.route('/profile')
@login_required
def profile():
    today = date.today()
    active_reservations = Reservation.query.filter_by(
        user_id=current_user.id,
        status='active').all()
    issued_reservations = Reservation.query.filter_by(
        user_id=current_user.id,
        status='issued').all()
    active_fines = Fine.query.filter_by(
        user_id=current_user.id,
        status='active').all()
    history_reservations = Reservation.query.filter(
        Reservation.user_id == current_user.id,
        Reservation.status == 'closed').order_by(Reservation.
        returned_at.desc()).limit(10).all()
    return render_template('profile.html',
        active_reservations=active_reservations,
        issued_reservations=issued_reservations,
        active_fines=active_fines,
        history_reservations=history_reservations,
        today=today)
```

HTML-код реализации страницы личного кабинета читателя

```
{% extends "base.html" %}
{% block title %}Личный кабинет - КнигоЛов{% endblock %}
{% block content %}
<div class="profile-header">
<div class="profile-avatar">
<i class="fas fa-user-circle"></i>
</div>
<div class="profile-info">
<h1>{{ current_user.last_name }} {{
current_user.first_name }}</h1>
<p class="profile-email">{{ current_user.email }}</p>
<div class="profile-meta">
{% if current_user.phone %}
<span><i class="fas fa-phone"></i> {{
current_user.phone }}</span>
{% endif %}
</div>
{% set fine_amount = namespace(value=0) %}
{% for fine in active_fines %}
{% set fine_amount.value = fine_amount.value +
fine.amount %}
{% endfor %}
{% if fine_amount.value > 0 %}
<div class="fine-warning">
<i class="fas fa-exclamation-triangle"></i>
```

```

У вас есть неоплаченный штраф на сумму <strong>{{
fine_amount.value }} ₸</strong>
</div>
{% endif %}
</div>
</div>
<div class="profile-sections">
<section class="profile-section">
<h2><i class="fas fa-bookmark"></i> Активные
бронирования</h2>
{% if active_reservations %}
<div class="reservations-grid">
{% for reservation in active_reservations %}
<div class="reservation-item">
<div class="reservation-book">
<h3>{{ reservation.book_obj.title }}</h3>
<p class="book-authors">
{% for author in reservation.book_obj.authors %}
{{ author.last_name }} {{ author.first_name[0] }}. {% if
not loop.last %}, {% endif %}
{% endfor %}
</p>
</div>
<div class="reservation-status">
<span class="status status-active">Ожидает
выдачи</span>
<p class="reservation-date">Забронировано: {{
reservation.created_at.strftime('%d.%m.%Y') }}</p>
</div>
</div>

```

```

{% endfor %}
</div>

{% else %}
<div class="empty-state-small">
<p>Нет активных бронирований</p>
</div>

{% endif %}
</section>

<section class="profile-section">
<h2><i class="fas fa-book"></i> Выданные книги</h2>
{% if issued_reservations %}
<div class="issued-books">
{% for reservation in issued_reservations %}
<div class="issued-book {% if reservation.due_date <
today %}overdue{% endif %}">
<div class="book-info">
<h3>{{ reservation.book_obj.title }}</h3>
<div class="book-meta">
<p><strong>Срок возврата:</strong> {{
reservation.due_date.strftime('%d.%m.%Y') }}</p>
<p><strong>Выдана:</strong> {{
reservation.issued_at.strftime('%d.%m.%Y') }}</p>
</div>
</div>
<div class="book-status">
{% if reservation.due_date < today %}
<div class="overdue-warning">
<i class="fas fa-exclamation-triangle"></i>
<span>Просрочена на {{ (today -
reservation.due_date).days }} дней</span>

```



```

</div>
{% else %}
<div class="days-left">
<span>Осталось дней: {{ (reservation.due_date -
today).days }}</span>
</div>
{% endif %}
</div>
</div>
{% endfor %}
</div>
{% else %}
<div class="empty-state-small">
<p>Нет выданных книг</p>
</div>
{% endif %}
</section>
<section class="profile-section">
<h2><i class="fas fa-exclamation-triangle"></i>
Активные штрафы</h2>
{% if active_fines %}
<div class="fines-list">
{% for fine in active_fines %}
<div class="fine-item">
<div class="fine-info">
<h3>{{ fine.fine_reservation_rel.book_obj.title }}</h3>
<p>{{ fine.reason }}</p>
<p class="fine-date">Начислен: {{
fine.created_at.strftime('%d.%m.%Y') }}</p>
</div>

```

```

<div class="fine-amount">
<span class="amount">{{ fine.amount }} ₸</span>
</div>
</div>
{% endfor %}
</div>
{% else %}
<div class="empty-state-small">
<p>Нет активных штрафов</p>
</div>
{% endif %}
</section>
<section class="profile-section">
<h2><i class="fas fa-history"></i> История
бронирований</h2>
{% if history_reservations %}
<div class="history-list">
<table class="table">
<thead>
<tr>
<th>Книга</th>
<th>Дата выдачи</th>
<th>Дата возврата</th>
<th>Статус</th>
</tr>
</thead>
<tbody>
{% for reservation in history_reservations %}
<tr>
<td>{{ reservation.book_obj.title }}</td>

```

```

<td>{{ reservation.issued_at.strftime('%d.%m.%Y') if
reservation.issued_at else '-' }}</td>
<td>{{ reservation.returned_at.strftime('%d.%m.%Y') if
reservation.returned_at else '-' }}</td>
<td><span class="status status-{{ reservation.status
}}">
{% if reservation.status == 'closed' %}Возвращена{%
endif %}
</span></td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
<div class="empty-state-small">
<p>История пуста</p>
</div>
{% endif %}
</section>
</div>
{% endblock %}

```